arXiv:2605.30370v3 [cs.NE] 9 Jun 2026

Updating the standard neuron model in artificial neural networks

Raul Mohedano[1*], Thomas Batard[2], Erik Velasco-Salido[1,3], Ramsses De Los Santos Mendoza[2], Jorge H. Martínez[2], Stacey Levine[4], Marcelo Bertalmío[1*]

[1]Spanish National Research Council (CSIC), Madrid, Spain.
[2]Center for Research in Mathematics (CIMAT), Guanajuato, Mexico.
[3]Universidad Autónoma de Madrid (UAM), Madrid, Spain.
[4]National Science Foundation (NSF), Alexandria, VA, USA.

*Corresponding author(s). E-mail(s): raul.mohedano@csic.es; marcelo.bertalmio@csic.es;

Abstract

From their inception in the 1950s, artificial neural networks (ANNs) started using the so-called point neuron model then prevalent in neuroscience, hoping that this analogy would allow for a better emulation of brain function. Over the years the neuroscience literature has shown that the point neuron model is too simplistic to properly represent many fundamental neural processes; however, the standard neuron model in ANNs still remains the same. Here we substitute it by a very recent model of cortical cells and demonstrate through theoretical analyses and experimental results how, simply by using a more realistic neural unit element without augmenting the number of parameters, the resulting ANNs offer a number of important advantages that include increases in expressivity, robustness and learning speed, and a reduction in memorization and the amount of training data needed.

Introduction

In the neuroscience of the mid 20th century, the prevalent view on neural computation was a simplistic abstraction [1, 2] currently known as the point neuron model, where it is assumed that each individual neuron linearly sums its inputs, i.e. that its dendrites are passive and act as mere "readers" of incoming signals, and after this linear filtering there follows a nonlinear process to generate the neuronal output. This approach was adopted to represent neurons in artificial neural networks (ANNs) when they were created in the late 1950s [3], with the belief that matching the brain starts with emulating its building block, the neuron [4]. To this day, neural units in ANNs still follow the same paradigm [5], and the cascade of linear-nonlinear stages in ANNs is referred to as the standard deep learning (DL) framework [6].

However, the ANN architectures that are currently most popular are poorly aligned with brain functionality [7–10] and have some critical limitations that do not affect living beings, such as requiring huge amounts of training data and being extremely sensitive to virtually unnoticeable perturbations of their input [11]. Our conjecture is the following: at the root of these and other fundamental constraints and drawbacks of ANNs lies the fact that the point neuron model is overly simplistic. It assumes that dendrites only act linearly, but since the late 1990s and early 2000s it has become clear that dendritic processing is not only much more complex, but also that complex dendritic computations are an indispensable element of key neural behaviors and brain functions [12–14]. Of note, an essential property of dendrites is that they can receive backpropagating action potentials (bAPs) initiated from the soma [15], and the importance of bAPs for the computations of dendrites, single cells and the whole network cannot be overstated [16–18]. There is currently a renewed interest in the research community in leveraging neuroscience to accelerate progress in AI [19], and a number of recent works consider more realistic dendritic models to improve the performance of ANNs [5, 20]. However, these approaches still fall under the standard DL framework, which is not adequate to represent the back and forth exchange between cell output and dendritic contributions caused by bAPs [21].

Very recently, a new model for neurons in the primary visual cortex that considers the interaction between dendritic nonlinearities and bAPs was shown to be able to explain a number of physiological results that challenge

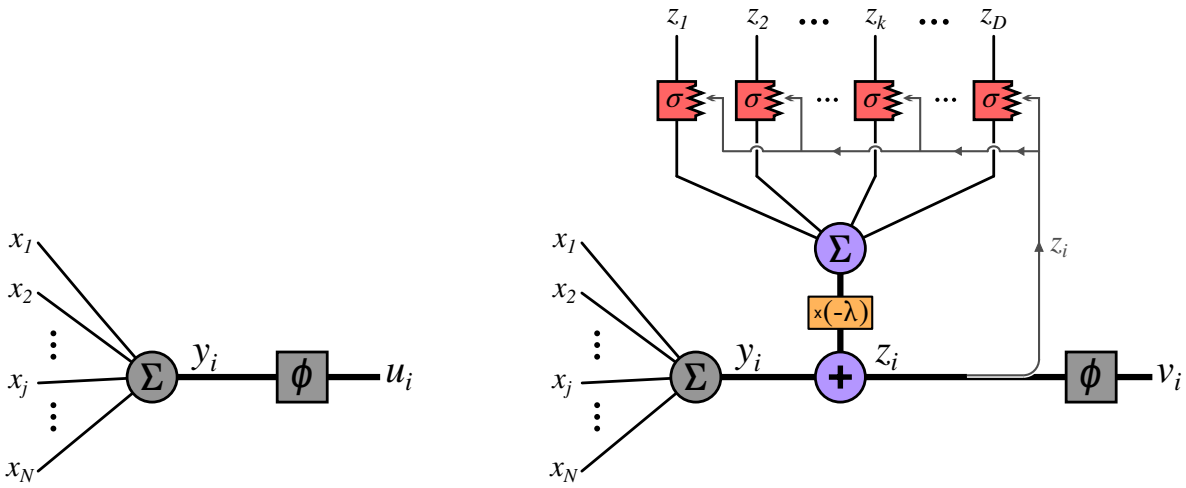


Fig. 1: Schematic neuron models. Left: standard neuron model in ANNs, following a simplistic abstraction from the neuroscience of the 1950s [1, 2] that assumed linear dendrites and an output nonlinearity. Right: proposed neuron model, based on a recent model of cortical cells [22]; notice how this new model extends the standard neuron model by considering internal nonlinearities σ (that represent dendritic processes), and these nonlinearities are affected in a dynamic way by the activity value z_i (that represents the bAP going from the soma to the dendrites), itself depending on the inputs x_j and the contributions z_k from other neurons.

standard approaches [22]. Here, we adapt this more realistic activity model of biological neurons into a novel formulation for the neural units in ANNs. Compared with an ANN that uses the standard neuron model and has the same number of parameters, an ANN built using this updated neuron model is much less impacted by the critical limitations mentioned above, being considerably more robust to input perturbations and requiring a significantly smaller amount of training data, as demonstrated by our theoretical analyses and experimental results. Furthermore, the novel neuron model allows the network to have other very relevant advantages as well: to be more expressive, requiring less parameters to achieve a given quality level in the task it solves; to learn faster, reaching a target performance with fewer learning steps; and to memorize less, which is an asset when the training data has noisy or mislabeled samples, as is generally the case.

Results

Updating the neuron model

We say that an ANN follows the standard model (SM) when the output u_i of a neural unit given an input $\mathbf{x} \in \mathbb{R}^N$ may be written as:

$$u_i = \phi(y_i), \tag{1}$$

$$y_i = \sum_{j=1}^{N} m_{ij} x_j - b_i, \tag{2}$$

where $\mathbf{m}_i \in \mathbb{R}^N$ is a linear filter, $b_i \in \mathbb{R}$ is the bias term, and ϕ is the (nonlinear) activation function.

Here we propose a new neuron model for ANNs that extends the standard model with the addition of a new bias term, inspired by a very recent neuroscience work [22], that emulates the contribution of dendritic nonlinearities mediated by their interaction with bAPs. In our formulation, the output v_i of a neural unit given an input $\mathbf{x} \in \mathbb{R}^N$ is

$$v_i = \phi(z_i), \tag{3}$$

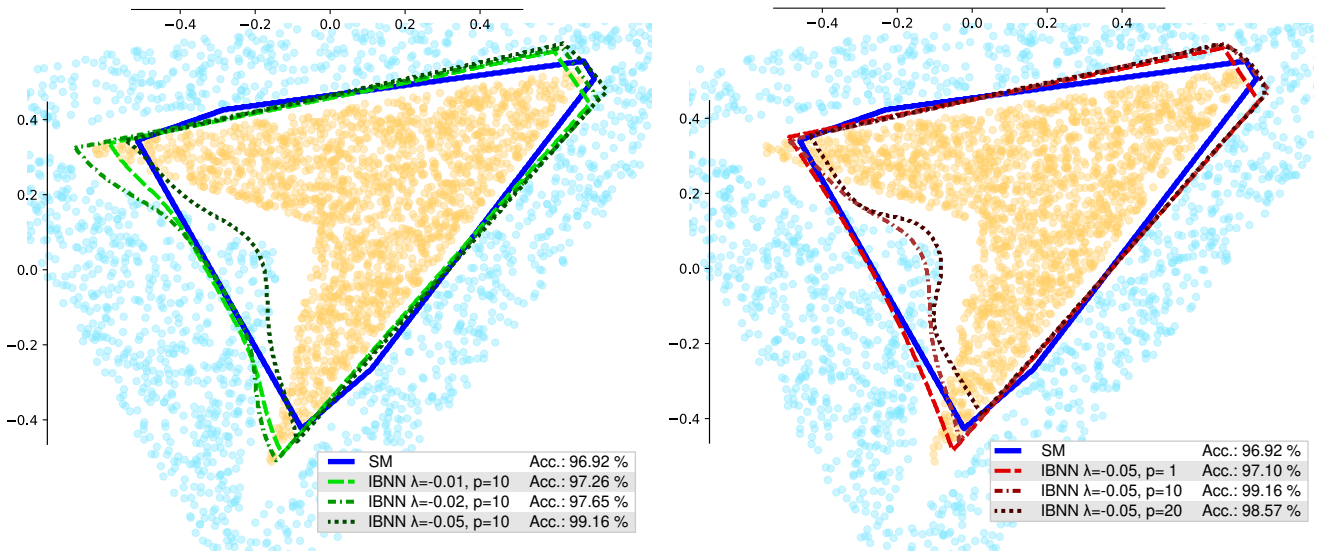
$$z_i = \overbrace{\sum_{j=1}^{N} m_{ij} x_j - b_i}^{y_i} - \overbrace{\lambda \sum_{k=1}^{D} w_{ik}\, \sigma(z_k - z_i)}^{\text{new bias}}, \tag{4}$$

where $\lambda \in \mathbb{R}$, $\mathbf{w}_i = (w_{ik})_{k=1}^{D} \in \mathbb{R}^D$ is a linear filter, and σ is a nonlinear function.

The first term in Eq. (4) represents the response to the input $\mathbf{x}$ produced by the linear dendrites; notice how this term is identical to y_i in Eq. (2), which is to be expected given that the SM only considers dendrites that are passive. The second term in Eq. (4) models the effect of nonlinear dendrites. In this new bias term, each of the dendrites has the nonlinear response function σ, which receives two inputs: the contribution z_k from a neighboring unit in the same neural layer, and the value z_i, which emulates the bAP from the soma. From Eqs. (2) and (4) it is trivial to see that, when $\lambda = 0$, our proposed neuron model is identical to the standard one; see Fig. 1 for an schematic of both models. Importantly, if we take λ and $\mathbf{w}_i$ to be fixed and predefined, then our proposed model has exactly the same number of trainable parameters as in the SM case, i.e. $\mathbf{m}_i$ and b_i.

Given that the new bias term involves z_i, the definition of z_i depends on z_i itself and therefore Eq. (4) is *implicit*. For this reason, we say that an ANN that uses our model is an *implicit bias neural network* (IBNN) . We can see from Eqs. (3) and (4) that computing the output of an IBNN requires solving a system of coupled implicit equations, and while this type of problem does not necessarily have a solution, in our case when σ is a sigmoidal function and $\lambda < T$ for some positive threshold T, the solution always exists, is unique, and minimizes an energy, and consequently it can be computed with a simple gradient descent algorithm (see Methods).

As an illustration, we consider a toy example in 2D where we want to find the boundary between two classes. We train different ANNs with analogous architecture to solve this task (see Methods): a SM network, and several IBNNs with a variety of values for the weight λ of the implicit bias term and the maximum slope of the nonlinearity σ, denoted by p. Fig. 2 shows the two classes in yellow and blue, and the classification boundaries obtained with the different ANNs. As we can see, IBNN is more expressive than the SM network, being able to approximate well the non-convex shape of the classification boundary while the latter is only capable of producing a convex polygon. Also, the resulting boundary in IBNN is able to become more curved as the magnitude of the parameter λ increases (Fig. 2, left), and p controls the curvature of the segments or arcs composing the classification boundary (Fig. 2, right).



(a) (b)

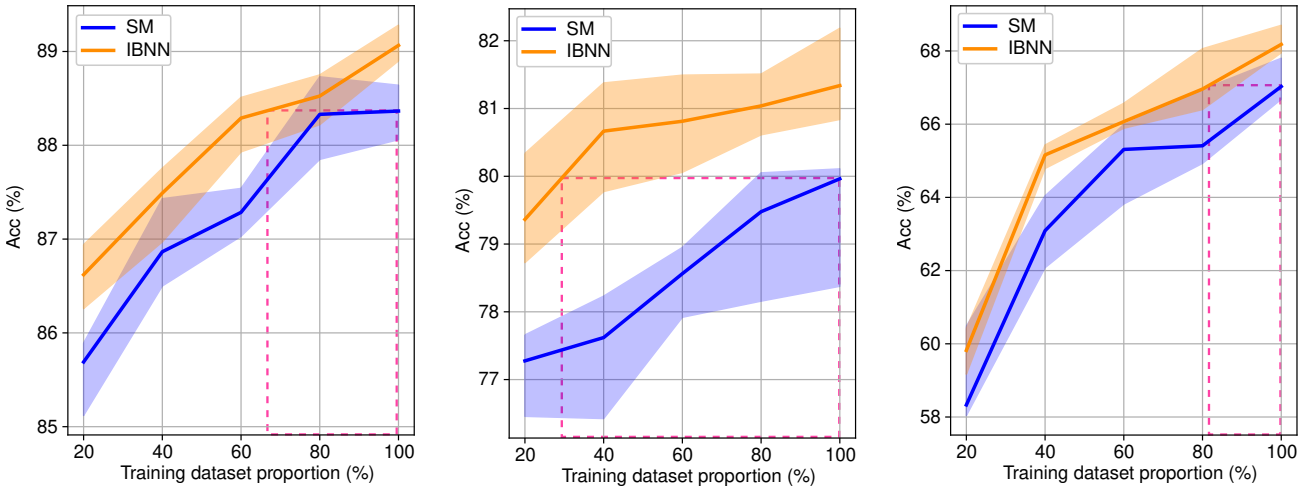
Fig. 2: Plot of the decision boundary for ANNs with a single hidden layer of three neurons in a binary classification of 2D input data: comparison between ANNs whose hidden layer is composed of neurons according to the SM(blue solid line in both left and right panels) and of neurons according to the bio-inspired neuron model with different weights λ for the implicit bias term (left panel, in shades of green) and different values of the maximum slope p of the sigmoidal nonlinearity σ (right panel, shades of red). The resulting validation accuracy of each ANN is included in the legend. Both panels suggest that the IBNN is more expressive than the SM network, as the IBNN is able to approximate well the non-convex shape of the classification boundary while the ANN using the SM neuron is only capable of producing a convex polygonal shape. The IBNN boundary can better fit curved boundaries as the magnitude of the parameter λ increases (left) while he maximum slope p of the nonlinearity σ controls the sharpness of the transitions between the segments or arcs composing the classification boundary (right).

Advantages of the networks that use the updated neuron model

We perform theoretical analyses and numerical experiments comparing in each case an ANN that uses SM neurons with an IBNN that has the same architecture and number of parameters but whose neurons follow the proposed model, thus ensuring that the benefits of IBNN do not come from an increase in model complexity. Numerical tests consist of solving image classification tasks over different image datasets and with different architectures, in order to highlight the generality of the approach.

Less training data required

Fig. 3 shows, for three different image datasets, that IBNN consistently requires just a fraction $(20-82\%)$ of the training data in order to achieve the best performance level of an ANN with neurons that follow the SM.



(a) Fashion-MNIST (b) SVHN (c) CIFAR-10

Fig. 3: IBNN consistently requires just a fraction $(29-82\%)$ of the training data in order to achieve the best performance level of an ANN with neurons that follow the SM. The pink dashed line indicates the best validation accuracy for SM and the proportion of the complete dataset needed to achieve it, and the proportion for the IBNN to achieve the same accuracy. Datasets are indicated under each graph and corresponding architectures indicated in Methods. Shadowed, CI of 90%.

Increased robustness to input perturbations

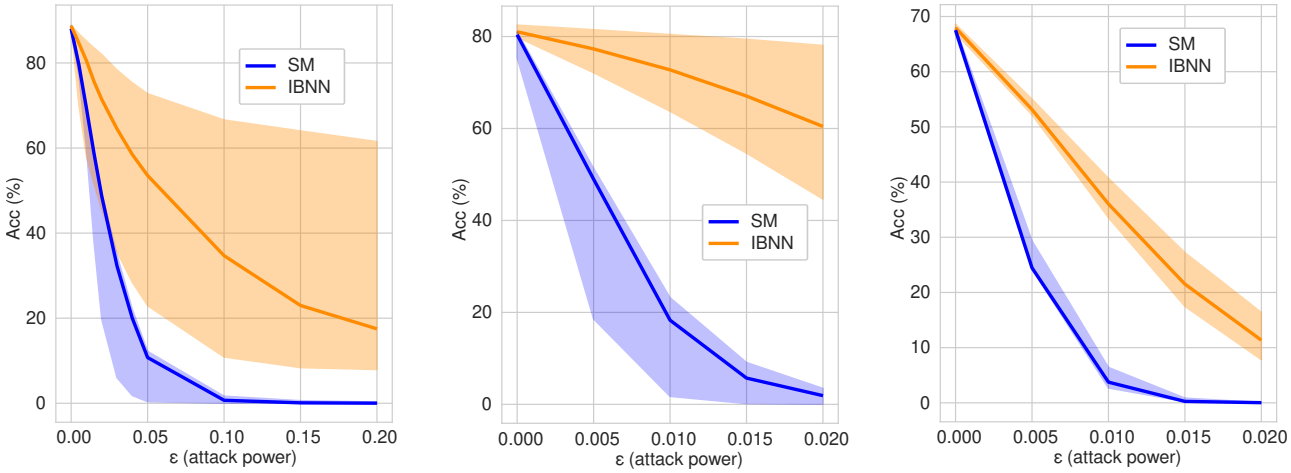
A common way to analyze the sensitivity of an ANN to input perturbations is to compute the norm of the Jacobian matrix of the network with respect to the input, with smaller norm values corresponding to more robust networks [23]. Under a mild set of assumptions, it can be shown that the norm of the Jacobian of an IBNN is smaller than that of its corresponding SM network (i.e. the network obtained by setting $\lambda = 0$), and therefore the IBNN is more robust (see Methods). It is notable that the formal mathematical proof presented in [6] to demonstrate that SM networks are either inaccurate or unstable (if accurate) does not directly apply to the IBNN formulation, whose structure does not satisfy the assumptions in [6] due to its implicit and highly nonlinear formulation (see Methods).

Recently, it has become very relevant to evaluate the robustness of networks against stealth attacks [24], defined as a scenario where an attacker modifies one or more neurons in an ANN in such a way that the following two conditions are met: (1) for inputs belonging to the validation set of the ANN, the responses are as expected, and therefore no attack is observed; (2) however, for some trigger input known only to the attacker, the ANN provides the result that the attacker wants. There is proof that a one-neuron stealth attack is possible on a SM network [24] but we demonstrate that it would be unfeasible on an IBNN (see Methods).

As experimental demonstration of the robustness capabilities that the proposed neuron model provides, Figs. 4 and 5 show comparisons of the performance of IBNN and SM networks with the same number of parameters against adversarial attacks of different nature. As we can see, IBNN is consistently less sensitive to input perturbations.

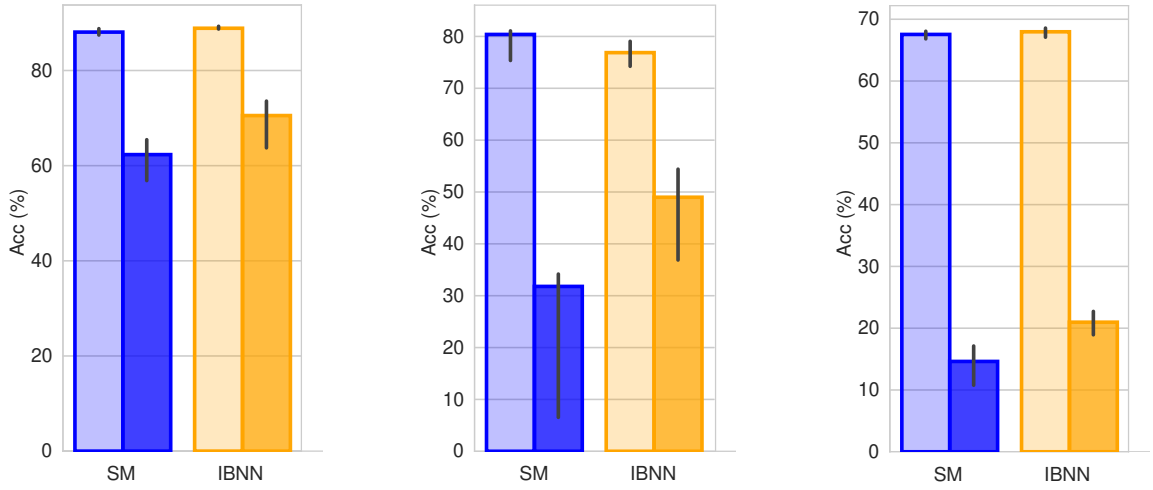
Greater expressivity

Assessing the expressivity of a neural network entails determining the functions it can compute. In analogy to the classical works that demonstrate the universal approximation power of ANNs based on the SM [27, 28], we



(a) Fashion-MNIST (b) SVHN (c) CIFAR-10

Fig. 4: IBNN is more robust to white-box adversarial attacks. The curves represent the accuracy achieved by each model for different intensities of an adversarial attack, in particular Projected Gradient Descent (PGD) [25] attack (power denoted in the horizontal axis: note that a power of 0 corresponds to the clean accuracy of the model). Datasets are indicated under each graph and corresponding architectures and details on the adversarial attacks are indicated in Methods. Shadowed, full range of data (CI of 100%).

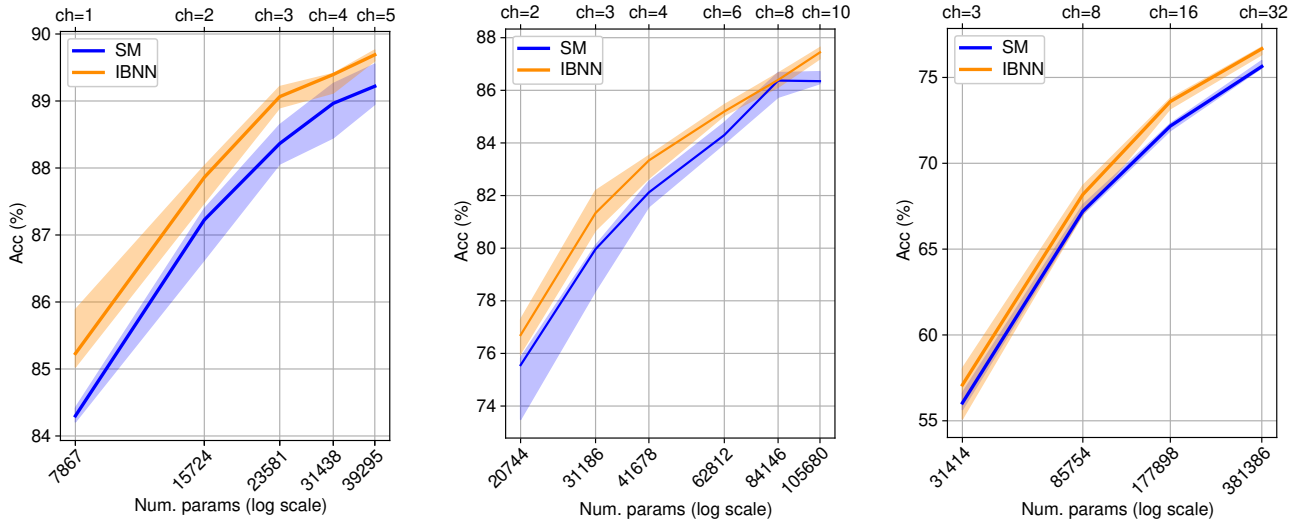


(a) Fashion-MNIST (b) SVHN (c) CIFAR-10

Fig. 5: IBNN is more robust to black-box adversarial attacks. The translucent bar represents clean accuracy for each model, whereas the solid bar represents the accuracy under an adversarial attack, in particular Pixle [26] of fixed strength. Datasets are indicated under each graph and corresponding architectures and details on the adversarial attacks are indicated in Methods. The black lines indicate the full range of obtained accuracies (CI of 100%).

establish that a single-layer IBNN is also able to approximate any given mapping with an arbitrarily small error (see Methods). But we also show that IBNNs are in fact more expressive than SM networks, because they need a smaller number of parameters to approximate the same function (see Methods).

As an example of the improvement in expressivity that the updated neuron model can provide to an ANN, Fig. 6 shows that the performance of an IBNN is consistently better than that of a SM network with the same number of trainable parameters or, equivalently, that IBNN can achieve the same performance as a SM network with less parameters. Note that the potential consideration of trainable λ would introduce an additional number of trainable parameters equal to the number of IBNN layers in the network, which would make a negligible difference to the comparison shown in the figure.

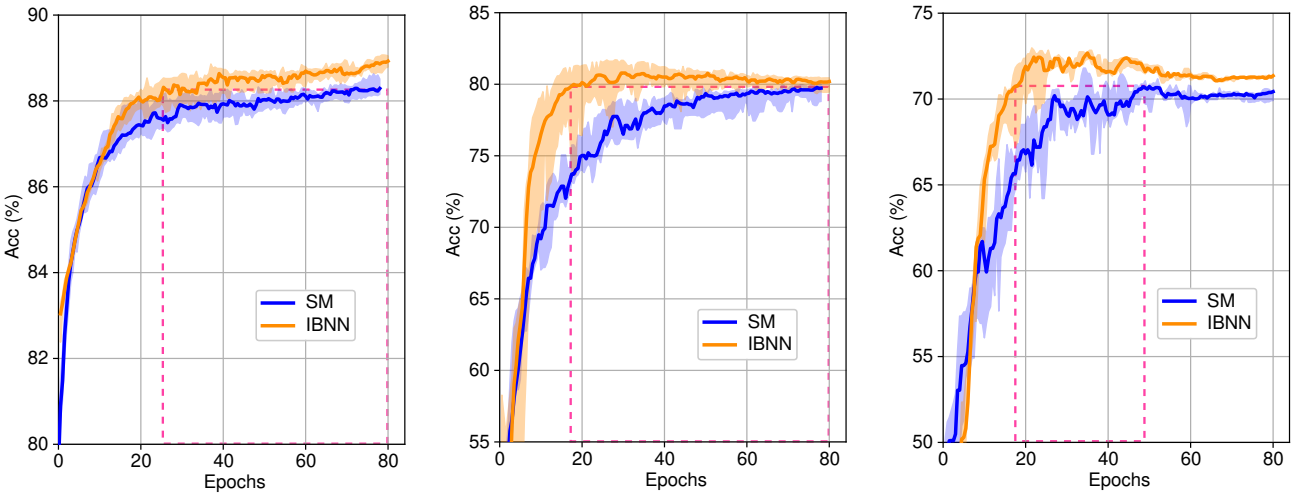


(a) Fashion-MNIST (b) SVHN (c) CIFAR-10

Fig. 6: IBNN is more expressive: it achieves higher accuracy with the same number of trainable parameters or, equivalently, it requires less parameters to perform as well as a SM network. Variation in the number for parameters caused by the increase in the number of channels of the layers of the backbone is accounted for in each graph. Datasets are indicated under each graph and corresponding architectures are indicated in Methods. Shadowed, CI of 90%.

Faster learning

Fig. 7 shows that IBNN requires less learning steps to achieve the same classification performance as a SM network: in all cases, this equated to less than half the number of epochs.



(a) Fashion-MNIST (b) SVHN (c) CIFAR-10

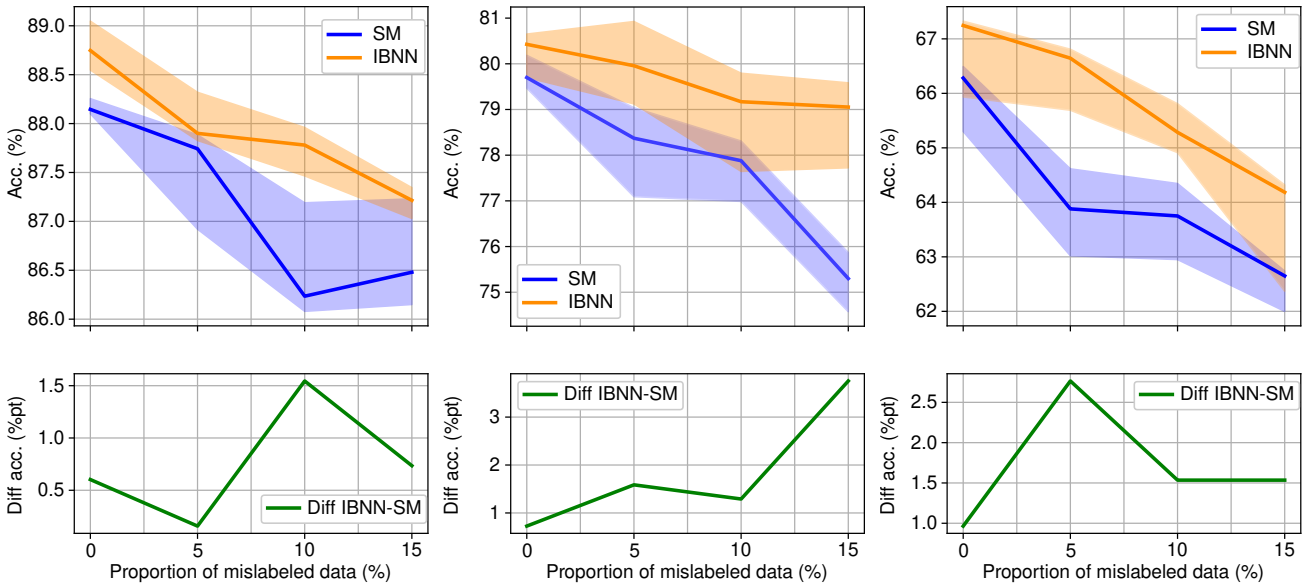
Fig. 7: IBNN learns faster: IBNN matches in 18-25 epochs the best validation accuracy reached by the SM network within a 80-epoch training, which, in all cases, happened after the 53-th epoch. The pink dashed line indicates the best validation accuracy for the SM over the considered training epochs and epoch when it was achieved, and the epoch for the IBNN to achieve the same accuracy. Datasets are indicated under each graph and corresponding architectures are indicated in Methods. Shadowed, CI of 90%.

Less memorization

When networks have a very large number of parameters they can manage to learn imperfect and even mislabeled training data, in a process called memorization [29]. This is clearly problematic, as the performance of the network

may become brittle. The occurrence of memorization correlates with the curvature of the loss function with respect to the input [30], and we provide a mathematical analysis suggesting that IBNN has a smaller curvature and therefore memorizes less than a SM network (see Methods).

Fig. 8 shows an experiment where the training data has been corrupted with varying percentages of mislabeled samples (details in Methods): the advantage of the best validation accuracy achieved by IBNN over the best validation achieved by an analogous SM network is consistently larger when the training set includes data with wrong labels, indicating that IBNN memorizes less.



(a) Fashion-MNIST (b) SVHN (c) CIFAR-10

Fig. 8: IBNN memorizes less: the improvement in validation accuracy of IBNN over the SM network is constently larger when the training set includes data with wrong labels and, in certain instances, the difference even widens as the proportion of wrong labels increases (see panel (b)). Datasets are indicated under each graph and corresponding architectures are indicated in Methods. For each dataset: top row, best validation accuracy achieved for each model for different percentage of wrong training labels; bottom row, difference of the validation of IBNN and SM depicted in the top row. Shadowed, CI of 90%.

Discussion

While ANNs continue to serve as the core of many state-of-the-art AI algorithms, they still have significant limitations that have not been overcome despite extensive research efforts. There appears to be an emerging consensus that the solution to these problems will not arrive simply from scaling up current ANNs, but that fundamentally new approaches are needed instead [19]. In this article we take inspiration from neuroscience and propose to use a new, compact and very effective model of cortical cells [22] to replace the omnipresent but clearly outdated point neuron model of ANNs. The novelty of the proposed approach lies in the combination of several bio-inspired elements, including internal dendritic-like nonlinearities that become dynamic and input-dependent through the modeling of the bAP, and lateral interactions among neurons that result in a system of coupled and implicit differential equations. In the NeuroAI literature there are recent works that introduce more involved models of neural units [5, 31–33], but they fail to emulate the bAP, or assume linear dendrites, or disregard lateral interactions. And, while some AI approaches use implicit layers and differential equations [34], they lack an equivalent to the bAP that might make the nonlinearities input-dependent.

The experimental and theoretical results presented in the article support our original hypothesis, namely, that the overly simplistic neuron model of ANNs contributes to their fundamental limitations, and that an updated and biologically inspired neuron model, one that takes into account the complexity of dendritic processing, may allow ANNs to achieve significant advantages. We have shown that ANNs using the new model demonstrate gains over a number of aspects that include expressivity, robustness and learning, and this aligns with the fact that nonlinear dendrites bring these same advantages to cortical computations and neural behaviors [14].

The experiments have focused on image data, and the model comes from visual neuroscience, but we expect the approach to generalize well, making extensions to other data modalities simple, in the same way that deep learning is applied to any sort of data despite its being based on the hierarchical model that Hubel and Wiesel proposed

for neurons in the visual cortex [2]. The IBNN parameters $\{\mathbf{m}_i, b_i\}$ in all experiments have been learned, but we have also tested a second mode of operation for our formulation where these parameters are simply taken from a SM network that solves the same task and has already been trained; the original mode produces more accurate results and is better suited for scenarios where training data is scarce (given that IBNN needs less training data), whereas the second mode can be seen as a fast and simple method to take an existing AI system and make it more robust to input perturbations, without the need to train it again (Supplementary Information).

There are several research questions related to the proposed model that are worth exploring as future work. To begin, we must note that in our experiments the optimum value for the hyperparameter λ was either found by grid search or through pre-training (i.e. by optimizing for accuracy); moving forward we believe that λ should always be considered a trainable parameter. In the same vein, the spatial extent of the filters $\mathbf{w}_i$ and the maximum slope p of the nonlinearity σ should also be learned. This would produce just a marginal increase in the number of parameters while it could enhance the expressivity of the network even further, although the mathematical analysis could become much more involved. Related to this, the use of values of λ above a certain positive threshold might imply the existence of multiple solutions for a single input, which could allow to represent with higher fidelity the modeling of biological processes through partial differential equations, where the final state of the process is often path-dependent [35]. We will also develop numerical techniques specifically tailored to IBNNs, in order to reduce the computational complexity associated to the iterative nature of finding the solution to the implicit system of equations, and also to the presence of multiple cross-comparisons in the nonlinear term of the new neuron model.

Secondly, while the proposed model was inspired by the V1 model in [22], for the sake of simplicity we did not emulate it fully, omitting one term of their formulation. We will incorporate that term to our model, which will introduce a non-monotonic internal nonlinearity, and analyze in which contexts this may provide an advantage (a recent work suggests that the use of non-monotonic nonlinearities might be key to predict time-varying phenomena [36]).

Thirdly, the second term of our model (i.e. the new bias in Eq. (3)) leads to the observation that IBNN has the ability to push for independent representations, because if the activities z_i and z_j are very similar then the neural units i and j do not influence each other; this is consistent with the fact that the V1 model that motivated our approach has a direct connection with the sparse coding framework [22]. We view this property of our proposed neuron model as providing a complementary behavior to the attention mechanism employed by Transformer architectures [37], it would be interesting to test if their combination (i.e. a Transformer using the updated neuron model) provides substantial gains in expressivity.

Finally, the mathematical form of our model, where the solution minimizes an energy and can be found through an iterative algorithm analogous to the numerical approximation of a differential equation (Methods, Eq. (12)), links our approach with energy models for AI [38] and also with causal modeling for machine learning [39], a topic of the utmost importance as it has been argued that the major unsolved problems in AI are fundamentally tied to causality [40].

Methods

Model solution

Existence, uniqueness, and energy minimization

We will next give some conditions on λ for which we have (i) existence and (ii) uniqueness of the solutions of Eq. (4). To that aim, we establish a connection between Eq. (4) and energy functionals of the form

$$E_\lambda(\mathbf{z};\mathbf{y}) = g(\mathbf{z};\mathbf{y}) - f_\lambda(\mathbf{z})\,, \mathbf{z},\mathbf{y} \in \mathbb{R}^D \text{ with } \begin{cases} g(\mathbf{z};\mathbf{y}) &= \dfrac{1}{2}\|\mathbf{z}-\mathbf{y}\|_2^2 \\ f_\lambda(\mathbf{z}) &= \dfrac{\lambda}{2}\displaystyle\sum_{i=1}^{D}\sum_{k=1}^{D} w_{ik} S(z_k - z_i) \end{cases}, \tag{5}$$

where

- S is a primitive function of σ, and
- w_{ik} are symmetric, positive, and normalized (i.e. $w_{ik} = w_{ki} \geq 0$ and $\sum_{k=1}^{N} w_{ik} = 1$), $0 < i, k \leq D$.

Proposition 1 *The solutions of Eq. (4), if they exist, are the critical points of $E_\lambda(\cdot; M\mathbf{x} - \mathbf{b})$ provided that S is even.*

Proof Denoting by $\nabla_{\mathbf{z}}$ the gradient operator with respect to $\mathbf{z}$, we have that

$$\nabla_{\mathbf{z}} g(\mathbf{z};\mathbf{y}) = \mathbf{z} - \mathbf{y}$$

Moreover, if S is even, then we have from the fact that $S' = \sigma$ and w are symmetric that

$$(\nabla_{\mathbf{z}} f_\lambda(\mathbf{z}))_i = -\lambda \sum_{k=1}^{D} w_{ik}\sigma(z_k - z_i),$$

from which we deduce that

$$(\nabla_{\mathbf{z}} E_\lambda(\mathbf{z};\mathbf{y}))_i = 0 \iff z_i = y_i - \lambda \sum_{k=1}^{D} w_{ik}\sigma(z_k - z_i).$$

□

Proposition 2 *If S is positive, even and satisfies that $S(z) \leq Az^2$, for some $A \geq 0$ then E_λ is coercive with respect to the first variable for any $\lambda < \frac{1}{4A}$.*

Proof We have

$$\frac{1}{2}\|\mathbf{z}-\mathbf{y}\|^2 \geq \frac{1}{2}\|\mathbf{z}\|^2 - \|\mathbf{z}\|\|\mathbf{y}\| + \frac{1}{2}\|\mathbf{y}\|^2 \tag{6}$$

from the Cauchy-Schwartz inequality.

As S is positive, the term $f_\lambda(\mathbf{z})$ in (5) is positive if $\lambda > 0$, from which follows that

$$E_\lambda(\mathbf{z};\mathbf{y}) \geq \frac{1}{2}\|\mathbf{z}\|^2 - \|\mathbf{z}\|\|\mathbf{y}\| + \frac{1}{2}\|\mathbf{y}\|^2 \qquad \text{if} \quad \lambda < 0,$$

which gives that E_λ is coercive with respect to $\mathbf{z}$ if $\lambda < 0$.

On the other hand, if $\lambda > 0$ and $S(z) \leq Az^2$, we have

$$\begin{aligned}
\frac{\lambda}{2}\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij} S(z_j - z_i) &\leq \frac{\lambda}{2} A \sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}(z_j - z_i)^2 \\
&\leq \frac{\lambda}{2} A \sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}(z_j^2 + z_i^2 + 2|z_j||z_i|) \\
&\leq \lambda A\|\mathbf{z}\|^2 + \lambda A \sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}|z_j||z_i| \qquad (7)
\end{aligned}$$

from Cauchy-Schwartz inequality.

Note that we have

$$\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij} z_i^2 = \sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij} z_j^2 = \|\mathbf{z}\|^2$$

as the kernel w is normalized. The Cauchy-Schwartz inequality applied to the vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^{D\times D}$:

$$a_{ij} = \sqrt{w_{ij}}|z_i| \qquad b_{ij} = \sqrt{w_{ij}}|z_j|$$

gives $\langle \mathbf{a}, \mathbf{b}\rangle \leq \|\mathbf{a}\|\|\mathbf{b}\|$, i.e.

$$\begin{aligned}
\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}|z_i||z_j| &\leq \left(\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij} z_j^2\right)^{1/2} \left(\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij} z_i^2\right)^{1/2} \qquad (8) \\
&\leq \|\mathbf{z}\|^2 \qquad (9)
\end{aligned}$$

We deduce from Eqs. (6), (7) and (8) that

$$\begin{aligned}
E_\lambda(\mathbf{z};\mathbf{y}) &\geq \frac{1}{2}\|\mathbf{z}\|^2 - \|\mathbf{z}\|\|\mathbf{y}\| + \frac{1}{2}\|\mathbf{y}\|^2 - \lambda A\|\mathbf{z}\|^2 - \lambda A\|\mathbf{z}\|^2 \\
&\geq \|\mathbf{z}\|^2\left(\frac{1}{2} - 2\lambda A\right) - \|\mathbf{z}\|\|\mathbf{y}\| + \frac{1}{2}\|\mathbf{y}\|^2.
\end{aligned}$$

We deduce that E_λ is coercive if $\lambda < \frac{1}{4A}$. □

Proposition 3 *If S is convex and twice differentiable with second order derivative bounded above by $K > 0$, then the energy in Eq. (5) is strictly convex with respect to the first variable for $\lambda < \frac{1}{2K}$.*

Proof We have

$$\frac{d}{dt}E_\lambda(\mathbf{z}+t\boldsymbol{\eta};\mathbf{y}) = \frac{1}{2}(2\langle \mathbf{z},\boldsymbol{\eta}\rangle + 2t\|\boldsymbol{\eta}\|^2 - 2\langle \boldsymbol{\eta},\mathbf{y}\rangle) - \frac{\lambda}{2}\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}(\eta_j-\eta_i)S'(z_j+t\eta_j-z_i-t\eta_i)$$

and

$$\frac{d^2}{dt^2}E_\lambda(\mathbf{z}+t\boldsymbol{\eta};\mathbf{y}) = \|\boldsymbol{\eta}\|^2 - \frac{\lambda}{2}\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}(\eta_j-\eta_i)^2 S''(z_j+t\eta_j-z_i-t\eta_i),$$

which gives

$$\frac{d^2}{dt^2}E_\lambda(\mathbf{z}+t\boldsymbol{\eta};\mathbf{y})_{|t=0} = \|\boldsymbol{\eta}\|^2 - \frac{\lambda}{2}\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}(\eta_j-\eta_i)^2 S''(z_j-z_i),$$

As S is twice differentiable and convex, we have, by definition, that $S'' \geq 0$. We deduce that $\frac{d^2}{dt^2}E_\lambda(\mathbf{z}+t\boldsymbol{\eta};\mathbf{y})_{|t=0} > 0$ if $\lambda < 0$, which shows that E_λ is strictly convex.

Let us now assume that $\lambda > 0$. We have that

$$\frac{d^2}{dt^2}E_\lambda(\mathbf{z}+t\boldsymbol{\eta};\mathbf{y})_{|t=0} \geq \|\boldsymbol{\eta}\|^2 - \frac{\lambda}{2}K\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}(\eta_j-\eta_i)^2.$$

Then, from the triangle inequality

$$(\eta_j-\eta_i)^2 \leq \eta_j^2 + \eta_i^2 + 2|\eta_j||\eta_i|,$$

we have

$$\frac{d^2}{dt^2}E_\lambda(\mathbf{z}+t\boldsymbol{\eta};\mathbf{y})_{|t=0} \geq \|\boldsymbol{\eta}\|^2 - \frac{\lambda}{2}K\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}(\eta_j^2+\eta_i^2) - \frac{\lambda}{2}K\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}|\eta_j||\eta_i| \tag{10}$$

The Cauchy-Schwartz inequality applied to the vectors $\mathbf{a},\mathbf{b} \in \mathbb{R}^{D\times D}$:

$$a_{ij} = \sqrt{w_{ij}}|\eta_i| \qquad b_{ij} = \sqrt{w_{ij}}|\eta_j|$$

gives $\langle \mathbf{a},\mathbf{b}\rangle \leq \|\mathbf{a}\|\|\mathbf{b}\|$, i.e.

$$\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}|\eta_i||\eta_j| \leq \left(\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}\eta_j^2\right)^{1/2}\left(\sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij}\eta_i^2\right)^{1/2} \leq \|\boldsymbol{\eta}\|^2 \tag{11}$$

We deduce from (10) and (11) that

$$\begin{aligned}\frac{d^2}{dt^2}E_\lambda(\mathbf{z}+t\boldsymbol{\eta};y)_{|t=0} &\geq \|\boldsymbol{\eta}\|^2 - \lambda K\|\boldsymbol{\eta}\|^2 - \lambda K\|\boldsymbol{\eta}\|^2 \\ &\geq \|\boldsymbol{\eta}\|^2(1-2\lambda K) \\ &\geq \|\boldsymbol{\eta}\|^2 > 0 \quad \text{if} \quad \lambda < \frac{1}{2K}.\end{aligned}$$

□

Corollary 3.1 *Under the assumption that S is positive, even and satisfies that $S(z) \leq Az^2$, for some $A \geq 0$, then Eq. (4) possesses a solution for $\lambda < \frac{1}{4A}$. Moreover, if S is also twice differentiable and convex with second order derivative bounded above by $K > 0$, then the solution is unique for $\lambda < \frac{1}{\max(4A,2K)}$.*

Proof According to Prop. 2, the energy $E_\lambda(\cdot; M\mathbf{x}-\mathbf{b})$ is coercive for $\lambda < \frac{1}{4A}$ if S is positive, even and satisfies that $S(z) \leq Az^2$. Together with the continuity of $E_\lambda(\cdot; M\mathbf{x}-\mathbf{b})$, it guarantees the existence of a minimum of $E_\lambda(\cdot; M\mathbf{x}-\mathbf{b})$. Then, by differentiability of $E_\lambda(\cdot; M\mathbf{x}-\mathbf{b})$, its minima correspond to the points vanishing its gradient, and these latter correspond to solutions of Eq. (4) according to Prop. 1. Moreover, if S is also twice differentiable and convex with second order derivative bounded above by $K > 0$, then the energy is also strictly convex if $\lambda < \frac{1}{2K}$ according to Prop. 3. We deduce that $E_\lambda(\cdot; M\mathbf{x}-\mathbf{b})$ is coercive and strictly convex for $\lambda < \min\left(\frac{1}{4A}, \frac{1}{2K}\right) = \frac{1}{\max(4A,2K)}$, which guarantees the uniqueness of the minimum of the energy, which shows the uniqueness of the solution of Eq. (4). □

Practical computation of the solution

Corollary 3.2 *Under the assumptions of Prop. 2 and for a well-chosen sequence* $\{\tau^{(n)}\}$*, the sequence* $\mathbf{z}^{(n)} = (z_i^{(n)})_{i=1}^D$ *defined by*

$$z_i^{(n+1)} = z_i^{(n)} + \tau^{(n)}\Big(-z_i^{(n)} + \sum_{j=1}^{N} m_{ij}x_j - b_i - \lambda \sum_{k=1}^{D} w_{ik}\sigma(z_k^{(n)} - z_i^{(n)})\Big), \qquad i \in \{1,\dots,D\}\,, \tag{12}$$

converges; and, for ϕ *continuous, the sequence* $\Phi(\mathbf{z}^{(n)}) = (\phi(z_i^{(n)}))_{i=1}^D \in \mathbb{R}^D$ *converges to an output of the corresponding IBNN layer. Moreover, under the assumptions of Corollary 3.1,* $\Phi(\mathbf{z}^{(n)})$ *converges to the unique output of the IBNN layer layer from any initial guess* $\mathbf{z}^{(0)} = (z_i^{(0)})_{i=1}^D$.

Proof Under the assumptions of Prop. 2, the energy $E_\lambda(\cdot; M\mathbf{x} - \mathbf{b})$ possesses a minimum as it is continuous and coercive. Moreover, the sequence (12) corresponds to the gradient descent algorithm associated to this energy, where $\tau^{(n)}$ is the step size at iteration n. Then, the convergence of $\mathbf{z}^{(n)}$ to a critical point of $E_\lambda(\cdot; M\mathbf{x} - \mathbf{b})$ is guaranteed if $\tau^{(n)}$ is well-chosen (e.g. $\tau^{(n)}$ satisfies the Wolfe conditions). As the critical points of $E_\lambda(\cdot, M\mathbf{x} - \mathbf{b})$ correspond to the solutions of Eq. (4), it shows that the sequence $\Phi(\mathbf{z}^{(n)})$ converges to an output of an IBNN layer if Φ is continous. Moreover, under the assumptions of Corollary 3.1, there is uniqueness of the critical point of $E_\lambda(\cdot, M\mathbf{x} - \mathbf{b})$, which guarantees the convergence of the sequence $\Phi(\mathbf{z}^{(n)})$ to the unique output of an IBNN layer. □

This iterative algorithm represents an intuitive and natural approach to the output of the IBNN layer. However, in the conditions where the solution of the system of coupled implicit equations of Eq. (4) exists and unique, the proposed layer is agnostic to the specific algorithm leading to such solution: the practical implementation of the layer could, therefore, be based on any fixed-point or root-finding algorithm able to efficiently solve the task, as discussed later in the context of the effective implementation used for the experimental evaluation.

Finally, we give a family of functions satisfying the desirable properties aforementioned.

Proposition 4 *The function*

$$S_p :\, z \mapsto \frac{\ln(\cosh(pz))}{p}\,,\ p \in \mathbb{R}^+ \tag{13}$$

satisfies $S_p(z) \leq \frac{p}{2}z^2$*, is positive, even, twice differentiable, and convex, and its second order derivative is bounded above by* p.

Proof It is clear that S_p is positive and even. Moreover, the trigonometric inequality $\cosh(z) \leq e^{\frac{z^2}{2}}$ [41] and the monotonicity of the logarithm yield

$$\frac{\ln(\cosh(pz))}{p} \leq \frac{p}{2}z^2\,.$$

Finally, the function S_p is clearly twice differentiable, and it is convex with second order derivative bounded above by p as

$$S_p'(z) = \tanh(pz) \quad (\triangleq \sigma(z)) \tag{14}$$

and

$$S_p''(z) = \frac{p}{\cosh^2(pz)},$$

from which we obtain that

$$0 < S_p''(z) \leq p.$$

□

Corollary 4.1 *The family of functions* $\sigma_p(z) = \tanh(pz)$ *fulfills the requirements of Corollaries 3.1 and 3.2 and therefore for the uniqueness of the solution of Eq. (4).*

IBNN in matrix notation

IBNN layers based on neurons according to Eqs. (3)-(4) can be written in a compact manner using multivariate functions and using matrix notation: we briefly define it next because it greatly clarifies certain proofs and results included in the following paragraphs.

- Let the affine function $\mathrm{A}:\mathbb{R}^N\to\mathbb{R}^D$ be
$$\mathrm{A}(\mathbf{x})=\mathrm{M}\mathbf{x}-\mathbf{b}\,, \tag{15}$$
with $\mathrm{A}\in\mathcal{M}_{D\times N}(\mathbb{R})$ and $\mathbf{b}\in\mathbb{R}^D$.
- Let the non-linear function $\mathrm{B}:\mathbb{R}^D\to\mathbb{R}^D$ be defined component-wise as $\mathrm{B}(\mathbf{z})=(\mathrm{B}_i(\mathbf{z}))_{i=1}^D$, where
$$\mathrm{B}_i(\mathbf{z})=\sum_{k=1}^{D} w_{ik}\,\sigma(z_k-z_i)\,. \tag{16}$$
- Let the function $\mathrm{F}:\mathbb{R}^D\to\mathbb{R}^D$ be implicitly defined, using B and with λ as a hyperparameter, as
$$\mathrm{F}(\mathbf{y};\lambda)=\{\mathbf{z}\mid\mathbf{z}=\mathbf{y}-\lambda\mathrm{B}(\mathbf{z})\}\,. \tag{17}$$
- And let $\Phi:\mathbb{R}^D\to\mathbb{R}^D$ be defined using a 1D component-wise outer activation ϕ as
$$\Phi(\mathbf{z})=(\,\phi(z_i)\,)_{i=1}^D\,. \tag{18}$$

Through these, Eqs. (3)-(4) defining IBNN can be condensed into

$$\mathrm{IBNN}=\Phi\circ\mathrm{F}(\cdot;\lambda)\circ\mathrm{A} \tag{19}$$

while SM can be condensed into

$$\mathrm{SM}=\Phi\circ\mathrm{A}\,. \tag{20}$$

Expressivity

Number of SM neurons required to reproduce the operation of an IBNN layer of D neurons

Due to the entanglement between their outputs and their implicit nature, the operations performed by IBNN layers cannot be efficiently represented using SM neurons, which act individually and whose operation is purely forward. To reinforce this idea, we will show that, even for infinitesimal λ, approximating the behavior of an IBNN layer of D-neurons could only be achieved, under the SM paradigm, with a number of neurons that is orders of magnitude higher.

To this end we will leverage the first-order Taylor expansion of the operation performed by the IBNN layer around $\lambda=0$. First, and using implicit differentiation on the matrix notation of Eq. (19), the derivative with respect to λ is

$$\frac{\partial\,\mathrm{IBNN}}{\partial\lambda}=\mathrm{D}\Phi\,\frac{\partial\mathrm{F}}{\partial\lambda},\qquad\text{with}\qquad\frac{\partial\mathrm{F}}{\partial\lambda}=-\Big(\mathrm{Id}_{D\times D}+\,\lambda\,\mathrm{DB}\Big)^{-1}\mathrm{B}\,, \tag{21}$$

which yields the expansion

$$\widetilde{\mathrm{IBNN}}(\mathbf{x})\;=\;\mathrm{IBNN}_{|\lambda=0}(\mathbf{x})+\lambda\,\frac{\partial\,\mathrm{IBNN}}{\partial\lambda}\bigg|_{\lambda=0}\;=\;\Phi\Big(\mathrm{A}(\mathbf{x})\Big)\;-\;\lambda\,\mathrm{D}\Phi\;\mathrm{B}\Big(\mathrm{A}(\mathbf{x})\Big)\,. \tag{22}$$

The calculation of the linearization (22) of the IBNN layer, composed of 2 summands, would require the following:

- The i-th component of the first summand would require the calculation of the vector multiplication (plus offset) $\mathrm{A}_i(\mathbf{x})=\mathbf{m}_i^T\mathbf{x}-b_i$ plus the application of the non-linearity ϕ, which coincides with operation of a SM neuron.
- The i-th component of the second summand would require the product of $\mathrm{B}_i(\mathrm{A}(\mathbf{x}))$ with a scalar factor, since the matrix $\mathrm{D}\Phi$ is diagonal. The calculation of $\mathrm{B}_i(\mathrm{A}(\mathbf{x}))$ (see Eq. (16)) entails calculations involving all the D components $\mathrm{A}_k(\mathbf{x})$, $k\in\{1,\dots,D\}$. In particular, and in line with the discussion in [31] for convolutional operations, (even if $\mathrm{A}(\mathbf{x})$ is presumed precalculated due to its implication in the first summand) the calculation of B_i requires, first, the calculation of all 'intermediate' signals $\sigma(\mathrm{A}_k(\mathbf{x})-\mathrm{A}_i(\mathbf{x}))$, $\forall k\in\{1,\dots,D\}$, requiring each a SM neuron with output nonlinearity σ, and then the application of another SM neuron (without output nonlinearity) to linearly combine them, resulting into $D+1$ neurons according to the SM.

Therefore, even for this first-order approximation of a layer of D IBNN neurons, which would neglect further nonlinear behavior that that the real IBNN layer would have, would require at least $D(D+2)$ neurons according to the SM paradigm.

A universal approximation theorem (UAT) for IBNN

A whole panoply of UATs regarding the ability of the SM-layers to approximate arbitrary functions exists [28, 42], differing on their respective assumptions about the type of network the SM layer is part of. The most widespread version of the UAT for the SM refers to networks composed of a single hidden SM layer with a linear combination of its outputs, that is, the class of functions

$$\mathcal{H}_{\mathrm{SM}} = \left\{ \mathbf{x} \mapsto \mathbf{c} \cdot \mathrm{SM}(\mathbf{x}; \mathrm{M}, \mathbf{b}) = \mathbf{c} \cdot \Phi(\mathrm{M}\mathbf{x} - \mathbf{b}) \right\}, \tag{23}$$

where $\Phi_i = \phi$ is a non-polynomial activation and where $\mathbf{x} \in X \subset \mathbb{R}^N$, X compact (note its correspondence with the SM neurons defined by Eqs. (1)-(2)), and its proof consist in showing that the set (23) is dense in the space of continuous functions $\mathcal{C}(X)$ [42]. (Note that we have explicitly included the matrix M and offset vector $\mathbf{b}$ in the notation of the SM in Eq. (23).)

We will analogously prove that the class of functions

$$\mathcal{H}_{\mathrm{IBNN}'} = \left\{ \mathbf{x} \mapsto \tilde{\mathbf{c}} \cdot \mathrm{SM}\left(\mathrm{IBNN}(\mathbf{x}; \mathrm{M}, \mathbf{b}, \lambda); \tilde{\mathrm{M}}, \tilde{\mathbf{b}} \right) = \tilde{\mathbf{c}} \cdot \Phi\left(\tilde{\mathrm{M}}\, \mathrm{IBNN}(\mathbf{x}; \mathrm{M}, \mathbf{b}, \lambda) - \tilde{\mathbf{b}} \right) \right\}, \tag{24}$$

again with a non-polynomial activation ϕ and defined for $\mathbf{x} \in K \subset \mathbb{R}^N$ compact, is dense in the space of functions $\mathcal{C}(X)$. Note that $\mathcal{H}_{\mathrm{IBNN}'}$ corresponds to an IBNN layer followed by a SM layer and a linear combination of the outputs of the latter.

Proposition 5 *For every $f \in \mathcal{C}(X)$ and every $\epsilon > 0$ there exists $\tilde{f} \in \mathcal{H}_{\mathrm{IBNN}'}$ such that*

$$\sup_{\mathbf{x} \in X} \left| f(\mathbf{x}) - \tilde{f}(\mathbf{x}) \right| \; < \; \epsilon \,.$$

Proof By the UAT for $\mathcal{H}_{\mathrm{SM}}$, for any given $f \in \mathcal{C}(X)$ there exist $\mathbf{c}$, M, and $\mathbf{b}$ such that the approximating function $g(\mathbf{x}) = \mathbf{c} \cdot \Phi(\mathrm{M}\mathbf{x} - \mathbf{b})$ satisfies

$$\sup_{\mathbf{x} \in X} \left| f(\mathbf{x}) - g(\mathbf{x}) \right| \; < \; \frac{\epsilon}{2} \,.$$

For this proof, let us assume that the nonlinearity ϕ is injective; although ReLU, the prototypical activation function and the basis for the experiments in this manuscript, is in fact not injective, we could assume, without any loss of generality for the purpose of this proof, a family of injective softplus functions with uniform convergence to it. This allows the definition and use of the left inverse $\Phi^{-1}_{\mathrm{left}}$ such that $\Phi^{-1}_{\mathrm{left}} \circ \Phi = \mathrm{Id}$. In addition, for λ where $\mathrm{F}(\cdot; \lambda)$ is single valued and is injective, the $\mathrm{L}(\cdot; \lambda) : \mathbb{R}^D \to \mathbb{R}^D$ such that

$$\mathrm{L}(\mathbf{z}; \lambda) = \mathbf{z} + \lambda \mathrm{B}(\mathbf{z}) \,, \tag{25}$$

is well defined and is, it and straightforwardly (see Eq. (17)) the left-inverse of $\mathrm{F}(\cdot; \lambda)$ and thus $\mathrm{L}(\cdot; \lambda) \circ \mathrm{F}(\cdot; \lambda) = \mathrm{Id}$. Using these two maps, we can define the function $\mathrm{H}(\cdot; \lambda) : \mathbb{R}^D \to \mathbb{R}^D$ as $\mathrm{H}(\cdot; \lambda) = \mathrm{L}(\cdot; \lambda) \circ \Phi^{-1}_{\mathrm{left}}$, which (represents by construction a partial, left inverse of the IBNN layer and) yields

$$\mathrm{H}\Big(\mathrm{IBNN}(\mathbf{x}; \mathrm{M}, \mathbf{b}, \lambda);\; \lambda \Big) = \mathrm{A}(\mathbf{x}) = \mathrm{M}\mathbf{x} - \mathbf{b} \,. \tag{26}$$

Let us define a new function $h(\mathbf{v}) = \mathbf{c} \cdot \Phi(\, \mathrm{H}(\mathbf{v}; \lambda)\,)$ based on the parameter $\mathbf{c}$ and the function Φ of the approximating function $g(\mathbf{x})$ above and on the function $\mathrm{H}(\cdot; \lambda)$ just defined. The domain of definition V of h is also compact, since $V = \mathrm{IBNN}(X)$ and IBNN is continuous. Additionally, h is continuous, since it is the composition of continuous maps. Leveraging again the UAT for the SM, this time on the compact domain V, there exist new $\tilde{\mathbf{c}}$, $\tilde{\mathrm{M}}$, and $\tilde{\mathbf{b}}$ such that

$$\sup_{\mathbf{v} \in V} \left| h(\mathbf{v}) - \tilde{\mathbf{c}} \cdot \Phi(\tilde{\mathrm{M}}\mathbf{v} - \tilde{\mathbf{b}}) \right| \; < \; \frac{\epsilon}{2} \,.$$

Since, using Eq. (26), we have

$$h\Big(\mathrm{IBNN}(\mathbf{x}; \mathrm{M}, \mathbf{b}, \lambda) \Big) = \mathbf{c} \cdot \Phi\left(\mathrm{H}\Big(\mathrm{IBNN}(\mathbf{x}; \mathrm{M}, \mathbf{b}, \lambda);\; \lambda \Big) \right) = \mathbf{c} \cdot \Phi(\mathrm{M}\mathbf{x} - \mathbf{b}) = g(\mathbf{x}) \,.$$

Therefore, the function $h \circ \mathrm{IBNN}(\cdot; \mathrm{M}, \mathbf{b}, \lambda)$ yields the claim through the triangle inequality. □

Comparative analysis of the robustness of IBNN

Reduced sensitivity of IBNN with respect to input perturbations

The following analysis, based on the matrix notation of Eqs. (15)-(20), compares the behavior of IBNN and SM layers regarding their sensitivity to input perturbation on the assumption that IBNN and SM nets having identical affine transforms (Eq. (15)) and scoring head serve to the same classification problem (Supplementary Information).

Local perturbations

What follows demonstrates that the output of the layers and of the classifier ANNs based on the newly proposed IBNN neurons are less sensitive to infinitesimal perturbations of their inputs than their SM counterparts, by virtue of the analysis of their Jacobian matrix.

Using implicit derivation [34] under the assumption that $\mathrm{F}(\cdot;\lambda)$ (see Eq. (15)) is a differentiable function $\mathbf{z}(\mathbf{y}) = \mathrm{F}(\mathbf{y};\lambda)$ of its input $\mathbf{x}$ we can write

$$\frac{\mathrm{d}}{\mathrm{d}\mathbf{y}}\,\mathbf{z}(\mathbf{y}) = \mathrm{Id}_{D\times D} - \lambda \frac{\mathrm{dB}}{\mathrm{d}\mathbf{z}}\frac{\mathrm{d}}{\mathrm{d}\mathbf{y}}\,\mathbf{z}(\mathbf{y})\,. \tag{27}$$

Reordering the terms and using the more compact symbol $\mathrm{D}(\cdot)_{|(\cdot)}$ to denote the absolute derivative calculated at the point indicated by its subscript yields

$$\frac{\mathrm{d}\mathbf{z}}{\mathrm{d}\mathbf{y}} = \mathrm{DF}_{|\mathbf{y}} = \left(\mathrm{Id}_{D\times D} + \lambda\,\mathrm{DB}_{|\mathrm{F}(\mathbf{y})}\right)^{-1} \triangleq \mathrm{K}_{\infty|\mathrm{F}(\mathbf{y})}\,. \tag{28}$$

In the following we will refer to this term as $\mathrm{DF}_{|\mathbf{y}}$ or $\mathrm{K}_{\infty|\mathrm{F}(\mathbf{y})}$.

Using Eq. (28), the Jacobian matrix of IBNN can be written through the application of the chain rule to Eq. (19) as

$$\mathrm{D\,IBNN}_{|\mathbf{x}} = \mathrm{D}\Phi_{|(\mathrm{F}\circ\mathrm{A})(\mathbf{x})}\;\mathrm{K}_{\infty|(\mathrm{F}\circ\mathrm{A})(\mathbf{x})}\;\mathrm{DA}\,, \tag{29}$$

where we have used that $\mathrm{DA}_{|\mathbf{x}} = \mathrm{DA} = \mathrm{M}$, constant for the whole input space. Note that, based on the vector formulation of Eq. (15), the Jacobian matrix of SM is

$$\mathrm{D\,SM}_{|\mathbf{x}} = \mathrm{D}\Phi_{|\mathrm{A}(\mathbf{x})}\;\mathrm{DA} = \mathrm{D}\Phi_{|\mathrm{A}(\mathbf{x})}\;\mathrm{Id}_{D\times D}\;\mathrm{DA}\,, \tag{30}$$

which highlights the analogies between $\mathrm{D\,SM}_{|\mathbf{x}}$ and $\mathrm{D\,IBNN}_{|\mathbf{x}}$ but also the decisive interposed term $\mathrm{K}_{\infty|(\mathrm{F}\circ\mathrm{A})(\mathbf{x})}$ present in the expression Eq. (29) of the latter. For convenience and for the sake of clarity, we will henceforth drop the explicit indication of the calculation point of each derivative.

Taking a deeper look into the shape and behavior of K_∞, we have that

$$\mathrm{DB} = \left(\mathrm{DB}_{ij}\right)_{i,j\in\{1,\dots,D\}}, \quad \text{with} \quad \mathrm{DB}_{ij} = \begin{cases} -\displaystyle\sum_{\forall k\neq i}\Big[w_{ik}\;\sigma'(z_k - z_i)\Big] & j = i \\ w_{ij}\;\sigma'(z_j - z_i) & j \neq i \end{cases}, \tag{31}$$

which means that, by virtue of the assumptions $w_{ji} = w_{ij}$ and σ odd, DB is symmetric, which in turn implies that it is diagonalizable and that it can be written as $\mathrm{DB} = \mathrm{Q}\,\Xi\,\mathrm{Q}^T$ for a certain real orthogonal matrix Q and a real diagonal matrix $\Xi = \mathrm{diag}\{\xi_1,\dots,\xi_D\}$. The fact that, additionally, each element in the main diagonal of DB is exactly the negative sum of the rest of elements of its row has interesting additional implications:

- Columns are linearly dependent, and thus at least one eigenvalue is 0 (and its corresponding eigenvalue is in fact the uniform vector $\frac{1}{\sqrt{D}}(1,\dots,1)$).
- Since w_{ik} and σ' are non-negative, DB is a symmetric diagonally dominant matrix with real non-positive diagonal elements, which makes it negative semidefinite and implies that $\xi_i \leq 0$, $\forall i \in \{1,\dots,D\}$. (In fact, by virtue of Gershgorin circle theorem, all the eigenvalues are within the interval $\xi_i \in [\,-2\max_i|\mathrm{DB}_{ii}|, 0\,]$.

These implications result (ordering eigenvalues in decreasing order of magnitude ($|\xi_1|\geq\ldots|\xi_i|\geq|\xi_D|$) without any loss of generality) in $\Xi = \mathrm{diag}\{\xi_1,\,\ldots,\,\xi_i,\,\ldots,\,\xi_{D-1},\,0\}$, where $\xi_i \leq 0$, $\forall i \in \{1,\dots,D-1\}$. The injection of the above properties of DB into the expression of K_∞, including $\xi_D = 0$, yields

$$\begin{aligned} \mathrm{K}_\infty &= \left(\mathrm{Id}_{D\times D} + \lambda\,\mathrm{Q}\,\Xi\,\mathrm{Q}^T\right)^{-1} = \mathrm{Q}\left(\mathrm{Id}_{D\times D} + \lambda\,\Xi\right)^{-1}\mathrm{Q}^T \\ &= \mathrm{Q}\;\mathrm{diag}\left\{\frac{1}{1+\lambda\xi_1}\,,\;\cdots\,,\;\frac{1}{1+\lambda\xi_i}\,,\;\cdots\,,\;\frac{1}{1+\lambda\xi_{D-1}}\,,\;1\right\}\;\mathrm{Q}^T\,. \end{aligned} \tag{32}$$

This expression directly represents the diagonalization of K_∞ and links its eigenvalues k_i one-to-one to the from the respective eigenvalues ξ_i of DB as $k_i = \frac{1}{1+\lambda\xi_i}$.

Since, as previously discussed, $\xi_i \leq 0$, $\forall i \in \{1,\dots,D-1\}$, we can conclude that any $\lambda \leq 0$ would ensure that all the eigenvalues k_i of K_∞ are in the range $(0, 1]$ and thus $|k_i|\leq 1$. For $\lambda > 0$, however, $|k_i|\leq 1$ is only fulfilled depending on the relative values of ξ_i and λ, and in particular for $\lambda \geq -\frac{2}{\xi_i}$. In both $\lambda > 0$ and $\lambda < 0$ cases K_∞ has at least one eigenvalue exactly 1.

The norm $\|\mathrm{K}_\infty\|_2$, representing the worst-case direction regarding vector stretch, is 1; nevertheless, the directions corresponding to eigenvectors of eigenvalues strictly $\xi_i < 0$ are compressed by the effect of K_∞. This implies that, $\forall\delta\mathbf{y} \in \mathbb{R}^D$, $\|\mathrm{K}_\infty\,\delta\mathbf{y}\|_2 = k\|\delta\mathbf{y}\|_2$ for some $k \in (0, 1]$ dependent on the specific $\delta\mathbf{y}$, which implies, in turn,

$$\begin{aligned}\|\mathrm{D\,IBNN}\ \delta\mathbf{x}\|_2 = &\ \|\mathrm{D}\Phi\ \mathrm{K}_\infty\ \mathrm{DA}\ \delta\mathbf{x}\|_2 \\ \leq &\ \|\mathrm{D}\Phi\|_2\ \|\mathrm{K}_\infty\ \mathrm{DA}\ \delta\mathbf{x}\|_2 \leq\ \|\mathrm{D}\Phi\|_2\ \|\mathrm{DA}\ \delta\mathbf{x}\|_2\,,\end{aligned} \tag{33}$$

where the second line results from the definition of the norm $\|\cdot\|_2$, ensuring $\|\Psi\,\delta\mathbf{x}\|_2 \leq \|\Psi\|_2\|\delta\mathbf{x}\|_2$. Finally, from this result and the relationship between Eq. (29) and (30), and under the assumption that both IBNN and SM share a similar A and are working under the same locally-linear region of the non-linear activation Φ and thus $\mathrm{D}\Phi_{|\mathrm{A}(\mathbf{x})} = \mathrm{D}\Phi_{|(\mathrm{F}\circ\mathrm{A})(\mathbf{x})}$, we have

$$\frac{\|\mathrm{D\,IBNN}\ \delta\mathbf{x}\|_2}{\|\delta\mathbf{x}\|_2} \leq \frac{\|\mathrm{D\,SM}\ \delta\mathbf{x}\|_2}{\|\delta\mathbf{x}\|_2}\,, \tag{34}$$

which would suggest, according to the reasoning in [23], a naturally improved robustness against adversarial attacks.

The above analysis for individual IBNN and SM layers also applies to networks having a backbone composed of a sequence of them. Suppose that we have respective IBNN- and SM-based backbones of L layers according to

$$\mathrm{IBNN}^{(1:L)} = \mathrm{IBNN}^{(L)} \circ \cdots \circ \mathrm{IBNN}^{(l)} \circ \cdots \circ \mathrm{IBNN}^{(1)}\,, \quad \text{with } \mathrm{IBNN}^{(l)} = \Phi \circ \mathrm{F}^{(l)}(\cdot;\lambda^{(l)}) \circ \mathrm{A}^{(l)} \tag{35}$$

and

$$\mathrm{SM}^{(1:L)} = \mathrm{SM}^{(L)} \circ \cdots \circ \mathrm{SM}^{(l)} \circ \cdots \circ \mathrm{SM}^{(1)}\,, \quad \text{with } \mathrm{SM}^{(l)} = \Phi \circ \mathrm{A}^{(l)}\,. \tag{36}$$

The application of the chain rule, along with Eq. (29) and (30), yields, respectively,

$$\begin{aligned}\mathrm{DIBNN}^{(1:L)} = &\ \mathrm{DIBNN}^{(L)} \cdots \mathrm{DIBNN}^{(l)} \cdots \mathrm{DIBNN}^{(1)} \\ = &\ \mathrm{D}\Phi^{(L)}\ \mathrm{K}_\infty^{(L)}\ \mathrm{DA}^{(L)} \cdots \mathrm{D}\Phi^{(l)}\ \mathrm{K}_\infty^{(l)}\ \mathrm{DA}^{(l)} \cdots \mathrm{D}\Phi^{(1)}\ \mathrm{K}_\infty^{(1)}\ \mathrm{DA}^{(1)}\end{aligned} \tag{37}$$

and

$$\begin{aligned}\mathrm{DSM}^{(1:L)} = &\ \mathrm{DSM}^{(L)} \cdots \mathrm{DSM}^{(l)} \cdots \mathrm{DSM}^{(1)} \\ = &\ \mathrm{D}\Phi^{(L)}\ \mathrm{Id}_{D\times D}\ \mathrm{DA}^{(L)} \cdots \mathrm{D}\Phi^{(l)}\ \mathrm{Id}_{D\times D}\ \mathrm{DA}^{(l)} \cdots \mathrm{D}\Phi^{(1)}\ \mathrm{Id}_{D\times D}\ \mathrm{DA}^{(1)}\,.\end{aligned} \tag{38}$$

For each IBNN layer with $\lambda^{(l)} \leq 0$ we have that, $\forall\delta\mathbf{y} \in \mathbb{R}^D$, $\|\mathrm{K}_\infty^{(l)}\,\delta\mathbf{y}\|_2 = k^{(l)}\|\delta\mathbf{y}\|_2$ for some $k^{(l)} \in (0,1]$ dependent on the specific $\delta\mathbf{y}$. Using sequentially this fact along with the definition of the norm $\|\cdot\|_2$ yields

$$\begin{aligned}\|\mathrm{DIBNN}^{(1:L)}\ \delta\mathbf{x}\|_2 &= \left\| \mathrm{D}\Phi^{(L)}\ \mathrm{K}_\infty^{(L)}\ \mathrm{DA}^{(L)} \cdots \mathrm{D}\Phi^{(l)}\ \mathrm{K}_\infty^{(l)}\ \mathrm{DA}^{(l)} \cdots \mathrm{D}\Phi^{(1)}\ \mathrm{K}_\infty^{(1)}\ \mathrm{DA}^{(1)}\ \delta\mathbf{x} \right\|_2 \\ &\leq \|\mathrm{D}\Phi^{(L)}\|_2 \left\| \mathrm{K}_\infty^{(L)}\ \mathrm{DA}^{(L)} \cdots \mathrm{D}\Phi^{(l)}\ \mathrm{K}_\infty^{(l)}\ \mathrm{DA}^{(l)} \cdots \mathrm{D}\Phi^{(1)}\ \mathrm{K}_\infty^{(1)}\ \mathrm{DA}^{(1)}\ \delta\mathbf{x} \right\|_2 \\ &= k^{(L)}\ \|\mathrm{D}\Phi^{(L)}\|_2 \left\| \mathrm{DA}^{(L)} \cdots \mathrm{D}\Phi^{(l)}\ \mathrm{K}_\infty^{(l)}\ \mathrm{DA}^{(l)} \cdots \mathrm{D}\Phi^{(1)}\ \mathrm{K}_\infty^{(1)}\ \mathrm{DA}^{(1)}\ \delta\mathbf{x} \right\|_2 \\ &\quad\ \vdots \\ &\leq\ k^{(L)} \cdots k^{(1)} \left(\|\mathrm{D}\Phi^{(L)}\|_2\ \|\mathrm{DA}^{(L)}\|_2 \cdots \|\mathrm{D}\Phi^{(1)}\|_2\ \|\mathrm{DA}^{(1)}\|_2\right) \|\delta\mathbf{x}\|_2\,.\end{aligned} \tag{39}$$

The fact that we have, analogously,

$$\begin{aligned}\|\mathrm{DSM}^{(1:L)}\ \delta\mathbf{x}\|_2 &= \left\|\mathrm{D}\Phi^{(L)}\ \mathrm{DA}^{(L)} \cdots \mathrm{D}\Phi^{(1)}\ \mathrm{DA}^{(1)}\ \delta\mathbf{x}\right\|_2 \\ &\leq \left(\|\mathrm{D}\Phi^{(L)}\|_2\ \|\mathrm{DA}^{(L)}\|_2 \cdots \|\mathrm{D}\Phi^{(1)}\|_2\ \|\mathrm{DA}^{(1)}\|_2\right) \|\delta\mathbf{x}\|_2\,,\end{aligned} \tag{40}$$

along with the fact that $k^{(l)} \leq 1$, $\forall l \in \{1,\ldots,L\}$, suggests that the sensitivity of the IBNN-based multilayer backbone to perturbations of its input is lower than its SM-based counterpart.

Global perturbations

What follows demonstrates that the output of the layers based on the newly proposed IBNN neurons are, globally, less affected by perturbations of their inputs than their SM counterparts. The demonstration stems from the functionals introduced to prove the existence and uniqueness of the solution of IBNN (Eq. (5), Methods).

Proposition 6 *Let* $\mathbf{x} \in \mathbb{R}^N$, $\mathcal{N}_\mathbf{x}$ *a neighborhood of* $\mathbf{x}$, *and* $\mathbf{y} \in \mathcal{N}_\mathbf{x}$. *Under the assumption that* $\lambda \leq 0$, *the function* S *in* f_λ *is convex, and* Φ *is a similarity in* $\mathcal{F} \subset \mathbb{R}^D$ *such that*

$$\mathcal{F} \supset \{\text{prox}_{f_\lambda}(\text{A}(\mathcal{N}_\mathbf{x}))\} \cup \{\text{A}(\mathcal{N}_\mathbf{x})\},$$

we have

$$\|\text{IBNN}(\mathbf{x}; \text{M}, \mathbf{b}, \lambda) - \text{IBNN}(\mathbf{y}; \text{M}, \mathbf{b}, \lambda)\| \leq \|\text{SM}(\mathbf{x}; \text{M}, \mathbf{b}) - \text{SM}(\mathbf{y}; \text{M}, \mathbf{b})\| \qquad \forall \mathbf{y} \in \mathcal{N}_\mathbf{x}.$$

Proof If $\lambda \leq 0$ and the function S in Eq. (5) is convex, then $-f_\lambda$ is convex. Together with the fact that $-f_\lambda$ is continuous and proper, we have that the variational problem $\underset{z}{arg\,min}\, E_\lambda(z; \text{A}(\mathbf{x}))$ is the proximal operator $\text{prox}_{-f_\lambda}(\text{A}(\mathbf{x}))$ of $-f_\lambda$ evaluated at $\text{A}(\mathbf{x})$, which leads to the existence and uniqueness of the solution of Eq. (4) and consequently of $\text{IBNN}(\mathbf{x}; \text{M}, \mathbf{b}, \lambda) \quad \forall\, \mathbf{x} \in \mathbb{R}^N$.

By property of proximal operators, we then have that

$$\|\text{prox}_{-f_\lambda}(\text{A}(\mathbf{x})) - \text{prox}_{-f_\lambda}(\text{A}(\mathbf{y}))\| \leq \|\text{A}(\mathbf{x}) - \text{A}(\mathbf{y})\| \qquad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^N.$$

We deduce that, under the assumption that Φ is a similarity, i.e. an isometry scaled by a constant, in $\mathcal{F} \subset \mathbb{R}^N$ such that

$$\mathcal{F} \supset \{\text{prox}_{-f_\lambda}(\text{A}(\mathcal{N}_\mathbf{x}))\} \cup \{\text{A}(\mathcal{N}_\mathbf{x})\},$$

we have

$$\|\Phi(\text{prox}_{-f_\lambda}(\text{A}(\mathbf{x})) - \Phi(\text{prox}_{-f_\lambda}(\text{A}(\mathbf{y}))\| \leq \|\Phi(\text{A}(\mathbf{x})) - \Phi(\text{A}(\mathbf{y}))\| \qquad \forall \mathbf{y} \in \mathcal{N}_\mathbf{x},$$

i.e.

$$\|\text{IBNN}(\mathbf{x}; \text{M}, \mathbf{b}, \lambda) - \text{IBNN}(\mathbf{y}; \text{M}, \mathbf{b}, \lambda)\| \leq \|\text{SM}(\mathbf{x}; \text{M}, \mathbf{b}) - \text{SM}(\mathbf{y}; \text{M}, \mathbf{b})\|.$$

□

Inconclusive evidence about accuracy-stability tradeoff, unlike for the SM

Bastounis et al. [6] demonstrated that multilayer SM-based ANNs with continuous nonlinear activations such as ReLU achieving a successful accuracy become universally unstable (see [6] Theorem 2.2 (ii)), which represents a formal proof that SM-based nets are subject to an inherent tradeoff between accurate and stability.

Their proof depends on the standard deep neural network structure Ψ whose L layers follow the form of Eqs. (1)-(2); that is, and using the notation of Eq. (15) and (18), it can be expressed as the composition $\Psi = \text{A}^{(L)} \circ \Phi \circ \text{A}^{(L-1)} \circ \cdots \circ \Phi \circ \text{A}^{(l)} \cdots \circ \Phi \circ \text{A}^{(1)}$, where $\Phi_i = \phi$ is a continuous nonlinear function applied coordinate-wise such as ReLU and $\text{A}^{(l)}$ are affine functions. A key factor in their proof depends on this standard network structure affording a linearization on finite but arbitrarily large sets ([6] lemma 5.5), a property which does not directly carry over to the implicit and highly nonlinear IBNN formulation. This linearization is critical in the construction of arbitrarily large sets where the output of the standard model neural network with fixed dimensions fails to match the true classification function output ([6] lemma 5.4). Thus the accuracy-stability tradeoff established in [6] does not directly apply to the IBNN.

IBNN is resistant to single-neuron stealth attacks

We define an ANN composed of one single hidden layer according to the IBNN paradigm and with uniform weights w_{ik} so

$$z_i(\mathbf{x}) = \mathbf{m}_i^T \mathbf{x} - b_i \; - \; \frac{\lambda}{D} \sum_{k=1}^{D} \sigma(z_k(\mathbf{x}) - z_i(\mathbf{x})) \tag{41}$$

$$v_i(\mathbf{x}) = \phi(z_i(\mathbf{x})) \tag{42}$$

and one single output layer so the output score of the resulting IBNN ANN is:

$$s(\mathbf{x}) = \sum_{i=1}^{N} c_i v_i(\mathbf{x}). \tag{43}$$

Following [24], we say that a successful single-neuron stealth attack on the IBNN ANN yielding the scoring function s would be a modified IBNN ANN with the same artificial units of the original plus an extra IBNN neuron and yielding a scoring function s_a such, given a verification input set $\mathcal{V}$, known to the owner of the system s but unknown to the attacker, and $\epsilon \geq 0$, $\Delta > 0$,

1. for some input $\mathbf{x}_T$, known to the attacker but not to the owner of the network, $|s_a(\mathbf{x}_T) - s(\mathbf{x}_T)| \geq \Delta$, and
2. for every input $\mathbf{x}_v \in \mathcal{V}$, $|s_a(\mathbf{x}_v) - s(\mathbf{x}_v)| \leq \epsilon$.

Condition (1) implies that the attacker makes the 'hacked' network s_a respond to a trigger input $\mathbf{x}_T$ in a way that is different from what the original s does, whereas condition (2) implies that the system owner will not be able to tell that an attack has taken place, that the original network s has been modified, because the responses of the hacked network s_a on $\mathcal{V}$ are very similar to those of the original network s.

The hacked network s_a is

$$s_a(\mathbf{x}) = \sum_{i=1}^{N} c_i \hat{v}_i(\mathbf{x}) + c_a \hat{v}_{N+1}(\mathbf{x}) \tag{44}$$

$$\hat{v}_i(\mathbf{x}) = \phi(\hat{z}_i(I)) \tag{45}$$

$$\hat{z}_i(\mathbf{x}) = \mathbf{m}_i^T \mathbf{x} - b_i \; - \; \frac{\lambda}{D} \sum_{k=1}^{D+1} \sigma(\hat{z}_k(\mathbf{x}) - \hat{z}_i(\mathbf{x})) \tag{46}$$

Let $\mathbf{x}_T$ be a trigger, such that the magnitude of the attack it produces is Δ; taking ϕ to be the ReLU function and assuming that the network units are working in the linear range of ϕ, we have

$$\Delta = |s_a(\mathbf{x}_T) - s(\mathbf{x}_T)| = \left| \sum_{i}^{D} c_i \Big(\hat{v}_i(\mathbf{x}_T) - v_i(\mathbf{x}_T) \Big) + c_a \hat{v}_{D+1}(\mathbf{x}_T) \right| . \tag{47}$$

If $D \gg 1$ we may assume that adding a single neuron does not change significantly the sum $\sum_{k=1}^{D} \sigma(z_j(\mathbf{x}) - z_i(\mathbf{x}))$, since only one term is added to the summation; therefore, we have $\hat{z}_i \simeq z_i$ and hence

$$\Delta \simeq |c_a \; \hat{z}_{D+1}(\mathbf{x}_T)| \; . \tag{48}$$

Additional, we have

$$\hat{z}_{D+1}(\mathbf{x}_T) = \mathbf{m}_{D+1}^T \mathbf{x}_T \; - \; \frac{\lambda}{D} \sum_{k=1}^{D+1} \sigma(\hat{z}_k(\mathbf{x}_T) - \hat{z}_{D+1}(\mathbf{x}_T)), \tag{49}$$

where, for simplicity, we have chosen $b_{D+1} = 0$. Combining Eqs. (48) and (49) we get

$$\left| \frac{\Delta}{c_a} \right| = \left| \mathbf{m}_{D+1}^T \mathbf{x}_T \; - \; \frac{\lambda}{D} \sum_{k=1}^{D+1} \sigma\Big(\hat{z}_k(\mathbf{x}_T) \pm \frac{\Delta}{c_a} \Big) \right| \tag{50}$$

$$\mathbf{m}_{D+1} = \frac{\mathbf{x}_T}{||\mathbf{x}_T||^2} (\pm \frac{\Delta}{c_a} \pm \frac{\lambda}{D} \sum_{k=1}^{D+1} \sigma(\hat{z}_k(\mathbf{x}_T) \pm \frac{\Delta}{c_a})) \tag{51}$$

$$\mathbf{m}_{D+1} \propto \mathbf{x}_T \tag{52}$$

This proves that we can find a filter $\mathbf{m}_{D+1}$ so that the input $\mathbf{x}_T$ produces very different outputs in s and s_a: the difference between the outputs of both networks has an amplitude of Δ, which would fulfill the first requirement of the stealth attack.

Now we need to check the second requirement, i.e. that the outputs for the verification set remain unaltered, or change very little for $\mathbf{x}_v \in \mathcal{V}$. Using analogous assumptions to the reasoning above, the difference Δ_v between $s_a(\mathbf{x}_v)$ and $s(\mathbf{x}_v)$ is

$$\Delta_v \simeq |c_a \; \hat{z}_{D+1}(\mathbf{x}_v)| \; . \tag{53}$$

We would like Δ_v to be negligible, e.g. $\Delta_v \simeq 0$, which gives $\hat{z}_{D+1}(\mathbf{x}_v) \simeq 0$, and from the definition of $\hat{z}_{D+1}$ we get

$$\mathbf{m}_{D+1}^T \mathbf{x}_v = \frac{\lambda}{D} \sum_{k=1}^{D} \sigma(z_k(\mathbf{x}_v)) \tag{54}$$

So, in order for the stealth attack to be actually stealth, undetectable, all the inputs in the verification set $\mathcal{V}$ should satisfy Eq. (54). But this is an implicit equation on $\mathbf{x}_v$, whose solutions, if any, might be totally unrelated to the elements of $\mathcal{V}$, which is a set that is only known to the system owner, not to the attacker.

In conclusion, condition (2) for the stealth attack is not satisfied, and therefore we can say that single-neuron stealth attacks on a IBNN are unfeasible.

Resistance to memorization

Here we show that IBNN is less prone to memorization by following the procedure suggested by Garg et al. [30], where the tendency of an ANN to memorize its training data is evaluated by computing the curvature of the loss function with respect to the input: lower curvature means less memorization.

Let Ψ be a binary classifier with a single hidden layer and a fully-connected (FC)-based scorer $s : \mathbb{R}^D \to \mathbb{R}$ according to

$$s(\mathbf{x}) = \mathbf{c}^T \ \mathrm{IBNN}(\mathbf{x}) - c_0 \,,$$

and a subsequent probability assignment according to $\Psi(\mathbf{x}) = P(s(\mathbf{x}))$, where $\mathbf{c} \in \mathbb{R}^D$, and $c_0 \in \mathbb{R}$ and where $P : \mathbb{R} \to [0, 1]$ is a function providing the probability of the class 1 (for two classes, with respective labels 0 and 1), e.g. a softmax summarization making y the probability of choosing the label 1, complementary of the probability of choosing the label 0. Additionally, let the loss for the training pair $(\mathbf{x}, y_t)$ (t for *truth*) be the Mean Squared Error (MSE), that is, $L(\mathbf{x}; y_t) = (\Psi(\mathbf{x}) - y_t)^2$. The derivative of the loss with respect to the input is, by virtue of the chain rule,

$$\frac{\partial L_{\mathrm{IBNN}}}{\partial \mathbf{x}} \ = \ 2\Big(\Psi(\mathbf{x}) - y_t\Big) \ P'(s(\mathbf{x})) \ \mathbf{c}^T \ \mathrm{D\,IBNN}_{|\mathbf{x}} \,. \tag{55}$$

An analogous expression $\frac{\partial L_{\mathrm{SM}}}{\partial \mathbf{x}}$ would correspond to an analogous SM-based network, *mutatis mutandis*.

The exact expression for $\mathrm{D\,IBNN}_{|\mathbf{x}}$ in Eq. (55) is provided by Eq. (29) — it is the product of the terms $\mathrm{D}\Phi_{|(\mathrm{F}\circ\mathrm{A})(\mathbf{x})}$, $\mathrm{K}_{\infty|(\mathrm{F}\circ\mathrm{A})(\mathbf{x})}$, and DA, which can be approximated as follows:

- $\mathrm{D}\Phi \approx \mathrm{Id}_{D\times D}$, assuming that the IBNN outer activation is a ReLU and that the network is working within its linear range;
- $\mathrm{K}_{\infty|(\mathrm{F}\circ\mathrm{A})(\mathbf{x})} \approx \mathrm{Id}_{D\times D} - \lambda\,\mathrm{DB}_{|\mathrm{A}(\mathbf{x})}$, assuming $|\lambda| \ll 1$, $(\mathrm{F}\circ\mathrm{A})(\mathbf{x}) \approx \mathrm{A}(\mathbf{x})$ and using the first-order Taylor expansion of Eq. (28).

Accordingly, $\mathrm{D\,IBNN}_{|\mathbf{x}}$ can be approximated as

$$\mathrm{D\,IBNN}_{|\mathbf{x}} \approx \Big(\mathrm{Id}_{D\times D} - \ \lambda\,\mathrm{DB}_{|\mathrm{A}(\mathbf{x})}\Big)\mathrm{DA} \tag{56}$$

which, in scalar notation and identifying $z_i = \mathrm{IBNN}_i(\mathbf{x})$, is

$$\frac{\partial z_i}{\partial x_j} \approx m_{ij} - \lambda \sum_k w_{ik}\sigma'\Big((\mathbf{m}_k^T - \mathbf{m}_i^T)\mathbf{x}\Big)(m_{kj} - m_{ij}) \,. \tag{57}$$

Assuming that σ is piece-wise linear with central slope p (i.e. $\sigma(z) = -1$, $\forall z < -\frac{1}{p}$; $\sigma(z) = 1$, $\forall z > \frac{1}{p}$; and $\sigma(z) = pz$, $\forall |z| < \frac{1}{p}$), that the weights w_{ik} are uniform with $w_{ik} = \frac{1}{D}$, $\forall i, k \in \{1, \ldots, D\}$, and that the norm of $\mathbf{m}_i^T\mathbf{x}$ does not vary significantly for different units i and thus $|(\mathbf{m}_k^T - \mathbf{m}_i^T)\mathbf{x}| \ll 1$, we can write

$$\frac{\partial z_i}{\partial x_j} \ \approx \ m_{ij} - \lambda p \left(\sum_{k=1}^{D} \frac{m_{kj}}{D}\right) + \lambda\, p\, m_{ij} = (1 + \lambda p)m_{ij} - \lambda p \left(\frac{1}{D}\sum_{k=1}^{D} m_{kj}\right) . \tag{58}$$

We now make the common assumption that the neurons have a reduced receptive field, i.e. each neural unit receives inputs from a small amount of coordinates of $\mathbf{x}$. Then, the matrix whose rows are the filters $\mathbf{m}_i^T$ is sparse, and with bounded filter elements (e.g. $|m_{ij}| \leq 1$, $\forall i, j$) we get $|\frac{1}{D}\sum_{j=1}^{D} m_{kj}| \ll 1$, which, plugged in Eq. (58), gives

$$\frac{\partial z_i}{\partial x_j} \ \approx \ (1 + \lambda p)m_{ij} \,, \tag{59}$$

where, just as a note, $\lambda = 0$ would correspond to the SM.

With this relationship between derivatives with respect to the input for IBNN and for SM, and going back to Eq. (55), we have

$$\frac{\partial L_{\mathrm{IBNN}}}{\partial x_j} \approx (1 + \lambda p)\frac{\partial L_{\mathrm{SM}}}{\partial x_j} \,. \tag{60}$$

Since, according to [30], the magnitude of the curvature of the loss function w.r.t. the input can be estimated as

$$\mathrm{Curv}(\mathbf{x}) \propto \left\| \frac{\partial L(\mathbf{x} + h\mathbf{g})}{\partial \mathbf{x}} - \frac{\partial L(\mathbf{x}))}{\partial \mathbf{x}} \right\|^2 , \tag{61}$$

where $\mathbf{g} \in \mathbb{R}^N$ are vectors sampled from a Rademacher distribution, the above approximation yields

$$\mathrm{Curv}_{\mathrm{IBNN}}(\mathbf{x}) = (1+\lambda p)^2 \, \mathrm{Curv}_{\mathrm{SM}}(\mathbf{x}) \tag{62}$$

Given that that the magnitude of the curvature is a measure of memorization [30], Eq. 62 shows that with a proper choice of (λ, p) (s.t. $|1+\lambda p| < 1$) IBNN memorizes less than a SM network.

Implementation of the IBNN layers

Trainable IBNN layers have been custom-implemented in Python using the design principles and the utilities provided by PyTorch [43] for both ANN training and evaluation. The implemented IBNN layers are analogous to and mostly interchangeable with the SM layers available in PyTorch, thus easing comparisons.

Two types of IBNN layers, convolutional and fully-connected (FC), have been implemented, which are, respectively, analogous to convolutional and FC layers according to the SM paradigm. The two types differ in the spatial limitations imposed on the linear filters represented by the elements $\{\mathbf{m}_i, b_i\}$ (see, respectively, Eq. (2) and (4)): convolutional layers have been achieved by shaping the filter elements $\{\mathbf{m}_i, b_i\}$ to correspond to a square mask and tying them across all the neurons of the same layer.

As highlighted before, the output $\mathbf{v}$ of an IBNN layer for a certain input $\mathbf{x}$ corresponds to the solution of a system of coupled implicit equations (Eqs. (3)-(4)), which represents a challenge for both forward and backward (i.e. training) operations. The library TorchDEQ [44], providing both operations, is the engine of the implicit calculations of our implementation of the IBNN layer:

- The forward operation of the IBNN layer, which is an implicit, but more specifically, fixed-point calculation, has been performed using the default forward solvers provided by TorchDEQ, in particular Broyden's method [44]. Additionally, a forward solver specifically tailored to IBNN and based on the minimization of Eq. (5) through the procedure of Eq. (12) has also been implemented. Neither showed convergence rates that would advise a preference for one over the other, and they have been used interchangeably.
- The backward operation, required for training, has been performed using the default forward solvers provided by TorchDEQ, in particular Broyden's method [44].

Our implementation of the IBNN layers allows for networks with independent λs at each layer, and for fixed or trainable λs: the latter option has been used in certain experiments as a sort of tentative hyperparameter optimization.

Experimental details

Datasets

The image classification experiments were based on three datasets of increasing difficulty level, each one containing 10 classes: Fashion-MNIST [45], SVHN [46], and CIFAR-10 [47]. These datasets have been used from their respective built-ins in the module Torchvision [48] of PyTorch [43]. Fashion-MNIST consists of 28×28 grayscale images of clothing, for a total of $60,000$ training and $10,000$ test images; their comprised classes are T-shirt/top, trousers, pullover, dress, coat, sandal, shirt, sneaker, bag, and ankle boot. SVHN consists of 32×32 color images of digits from 0 to 9, obtained as croppings of real street images, for a total $73,257$ training and $26,032$ test images. And CIFAR-10 consists of 32×32 color images of animals and vehicles, for a total of $50,000$ training and $10,000$ test images; their comprised classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

The visualizations in Fig. 2 correspond to a custom 2D dataset, generated using a custom defined mask containing the two concentric arrows hosting the 2D samples of the two separated classes: the samples of each class were generated by 'accept-reject sampling' from a 2D uniform distribution with a support comprising their respective masks. The resulting 2D samples where generated in the form of 2-pixel images for compatibility with the image-oriented implementation of the classifiers.

Network architectures used in the experimental evaluation

The classifier ANNs used in the presented experimental evaluation, which we will refer to as uniform convolutional networks (UCNs) and which will be denoted using the convention $\mathrm{UCN}(L, C)$, is a simplified architecture composed of:

- A backbone formed of L sequential blocks of analogous characteristics (hence the nomenclature 'uniform'). Each block $l \in \{1, \ldots, L\}$ contains a convolutional layer, defined according to the SM or IBNN depending on the case, and a batch normalization layer for its output. In all cases, the convolutional layer maintains (through zero padding) the spatial size of its input image but produces C channels at its output, and the side of the convolutional filters is in all cases the 15% of the size of its input image. ReLU is used as the output nonlinearity ϕ (Eq. (1) and (3)) of all layers.

- An output scoring head composed of one single FC layer (according to the SM) with as many neurons as classes contained in the dataset.

For convolutional layers according to IBNN, additionally, uniform weights $w_{ik} = \frac{1}{D}$ and a dendritic nonlinearity $\sigma(z) = \tanh(pz)$ with $p = 10$ is used. This value of p was chosen for two reasons: it is an intermediate value regarding boundary flexibility (see Fig. 2); and it ensures existence and uniqueness of IBNN in the considered ranges of λ (see Eq. (14)). The parameter λ_l of each layer $l \in \{1, \ldots, L\}$ is set or trained individually.

All experiments shown in Results correspond to the described UCN architecture. However, experiments corresponding to different datasets and aimed aspects differ in L and C. For each dataset, all experiments gravitate around a configuration UCN(L_d, C_d) whose L_d and C_d were set in order to achieve base accuracies in the range of $65-90\%$. This central configuration was, for each dataset: for Fashion-MNIST, UCN$(1, 3)$; for SVHN, UCN$(2, 3)$; and for CIFAR-10, UCN$(3, 8)$. All the experiments correspond to said central configurations but for two exceptions: experiments related to expressivity compare network sizes by varying the number of channels, and thus correspond for each dataset to the L indicated above but capture different values of C (see Fig. 6); and the result included in Fig. 7 for CIFAR-10, corresponding to UCN$(3, 16)$ is the only departure from these configurations: although both $C=8$ and $C=16$ similarly reproduce the same positive trend regarding learning speed, it was determined that the latter illustrates the concept better.

The visualizations in Fig. 2, however, correspond to a network with a backbone of a single hidden layer of FC type composed of 3 neurons (according to the SM or IBNN depending on the case) with a ReLU output nonlinearity and a batch normalization layer, followed by a single layer head/unit leading to a binary classification.

Training and evaluation details for IBNN and SM-based networks

Training of both SM- and IBNN-based ANNs was performed using Stochastic Gradient Descent (SGD). The adoption of SGD, instead of other more dynamic optimizers such as Adam, is motivated by two reasons: the latter requires the calculation of higher-order moments and therefore conveys a larger computational burden; and the simplicity of SGD has been regarded as more controllable by experimenters with respect to the more obscure optimization steps of the latter. A scheduler composed of a linear warmup from 0.001 to 0.01 in 40 epochs and a linear decay symmetric to the former was used for the learning rate (LR). This scheduler was chosen because it represents a favorable trade-off between stability with respect to the initial random weights of the network, reasonable speed due to the higher LR during the central epochs, and better convergence due to the final cooldown.

A warmup training step of 2 epochs was incorporated prior to the training of both SM- and IBNN-based ANNs. In the specific case of the IBNN-based nets, this warmup was performed on a SM-based surrogate with an identical architecture to that of the target IBNN; the training of the IBNN net itself starts by initializing its weights with the weights resulting from the warmup of the surrogate (see Eqs. (2) and (4), and consider also the batch normalization and head of the UCN). This surrogate-based warmup has been introduced to ensure favorable initializations of IBNN-based ANNs and to avoid those rare cases where random initializations without such warmup did not allow training to progress to viable classifiers.

Each model (labeled in each case as IBNN or SM) in each experiment in Results was obtained as the median of several instances of the same model, trained independently from distinct random initial weights and dataset reordering: the results addressing expressivity (Fig. 6) and memorization (Fig. 8) aggregate 10 instances for the SM nets and 5 for the IBNN nets; the results addressing decreasing percentages of the training data (Fig. 3) and learning speed (Fig. 7) aggregate 20 instances for the SM nets and 5 for the IBNN nets. Corresponding confidence intervals of 90% are included in all cases. Additionally, the results illustrating learning speed (Fig. 7) perform a running average of 3 epochs.

Adversarial attack generation and evaluation details

The respective adversarial robustness (AR) of SM- and IBNN-based nets has been assessed using two adversarial attacks (AAs) of very different nature to better characterize the abilities of the proposal: PGD [25] and Pixle [26]. In both cases, attacks have been generated using the library TorchAttacks [49] from the validation subset of each dataset.

Originally, PGD is a white-box attack that presumes the full availability of the target ANN and its gradient with respect to variations in its input. It performs a number of gradient-based iterations and limits its operation to a maximum distance with respect to the image from which it is calculated, denoted as ϵ. However, our implementation based on IBNN prevents the calculation of such a gradient and thus the feasibility of direct PGD attacks, analogous to the gradient masking/obfuscation defenses against AAs. Nevertheless, the literature has shown that attacks generated using differential surrogate models are usually effective for attack generation [50, 51]. Accordingly, while PGD attacks for the SM-based ANNs have been generated in a purely white-box manner, the PGD attacks for the IBNN-based ANNs have been produced using a SM-based surrogate model, as follows: each trained IBNN layer has been replaced by the SM layer resulting from simply removing the nonlinear bias of Eq. (4) (identical to

setting $\lambda = 0$ for the layer). This type of surrogate model has been selected because it is deemed to be the most likely surrogate chosen by a potential attacker due to its similarity to the target IBNN net.

Conversely, Pixle is purely black-box, working only on the classification probabilities reported by the ANN for a given query image, and has been used to directly attack both SM and IBNN without any need for surrogates. In this way, it provides a complementary view to the results obtained using PGD *via* a surrogate. Pixle gradually shuffles an increasing number of pixels from the original image as its so-called restarts advance, and optimizes the effectiveness of the newly scrambled pixels at each restart for a certain number of iterations.

All Pixle experiments presented in this manuscript correspond to an identical attack with a maximum number of 10 restarts with 5 iterations each, with the generation of pixel candidates based on similarity and from 3×3 image patches [26]. The PGD-based attacks presented in this manuscript perform 10 gradient iterations and sweep across ranges of attack intensities, from $\epsilon = 0$ to a certain maximum that depends on the dataset under study, resulting in the corresponding security evaluation curves [50] for each experimental condition.

The models subject to adversarial attacks were trained according to the same principles, architectures, and hyperparameters indicated above, and, as a result, several instances of each model were obtained and evaluated on the clean validation dataset and under attack. The results addressing both PGD (Fig. 4) and Pixle (Fig. 5) aggregate, using the median, 5 model instances for the SM nets and 5 for the IBNN nets. Since confidence interval estimation would be unreliable for the limited amount of available samples, adversarial attack experiments show the complete range of obtained results (equivalent to confidence intervals of 100%).

Memorization: experimental evaluation

Memorization occurs when models start learning individual training samples instead of the underlying statistical distribution of data: therefore, its occurrence must become apparent on the evolution of the network on its validation dataset.

We devised, consequently, the following experiment: we deliberately contaminated the training subset of a certain dataset with different proportions of wrong samples, generated by simply scrambling their labels, while keeping the validation subset unscathed; for each considered proportion we trained networks using the contaminated data and took the epoch when the corresponding validation accuracy started to permanently decline as a sign of memorization and overfitting; and we regard such maximum validation accuracy as the level of generalization achievable for an ANN on a contaminated dataset.

Code availability

The code used to carry out the simulations and analysis is available at https://github.com/vmg-io-csic/ibnn.

Author contributions

Conceptualization and methodology: M.B.; mathematical analysis: R.M., T.B., M.B., S.L.; numerical implementation: R.M., E.V.S.; experiments: R.M., E.V.S., R.S.M., J.H.M.; analysis of results: all authors; writing: R.M., M.B., T.B., S.L.

Acknowledgments

This work has been supported by: Programa Fundamentos (ref. VIS4NN) of Fundación BBVA 2022 (Spain); Programa Tecnologías (ref. TEC 2024/COM-322) of Comunidad de Madrid (Spain); Proyectos de Generación de Conocimiento 2024 (ref. PID2024-161841NB-I00) of Ministerio de Ciencia, Innovación y Universidades (Spain); and Programa Momentum (ref. MMT24-IO-02) of Consejo Superior de Investigaciones Científicas (Spain), funded by European Commission–NextGenerationEU through the Spanish Recovery, Transformation and Resilience Plan.

Additional information

Supplementary information

The present manuscript refers to Supplementary Information included as an appendix.

References

[1] McCulloch, W. S. & Pitts, W. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics* **5**, 115–133 (1943).

[2] Hubel, D. H. & Wiesel, T. N. Receptive fields of single neurones in the cat's striate cortex. *The Journal of Physiology* **148**, 574–591 (1959).

[3] Rosenblatt, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review* **65**, 386 (1958).

[4] Haykin, S. *Neural networks and learning machines* (New York: Prentice Hall, 2009).

[5] Chavlis, S. & Poirazi, P. Dendrites endow artificial neural networks with accurate, robust and parameter-efficient learning. *Nature Communications* **16**, 943 (2025).

[6] Bastounis, A., Hansen, A. C. & Vlačić, V. The mathematics of adversarial attacks in AI – Why deep learning is unstable despite the existence of stable neural networks. Preprint at https://arxiv.org/abs/2109.06098 (2021).

[7] Sartzetaki, C., Roig, G., Snoek, C. G. M. & Groen, I. International Conference on Learning Representations (ed.) *One hundred neural networks and brains watching videos: Lessons from alignment.* (ed.International Conference on Learning Representations) *Proceedings of the International Conference on Learning Representations* (2025).

[8] Kar, K. & DiCarlo, J. J. The quest for an integrated set of neural mechanisms underlying object recognition in primates. *Annual Review of Vision Science* **10**, 91–121 (2024).

[9] Bowers, J. S. *et al.* Deep problems with neural network models of human vision. *Behavioral and Brain Sciences* 1–74 (2022).

[10] Tong, S. *et al.* IEEE/CVF Computer Vision Shifts and Pattern Recognition Organizer Committee (ed.) *Eyes wide shut? Exploring the visual shortcomings of multimodal LLMs.* (ed.IEEE/CVF Computer Vision Shifts and Pattern Recognition Organizer Committee) *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 9568–9578 (2024).

[11] Wichmann, F. A. & Geirhos, R. Are deep neural networks adequate behavioral models of human visual perception? *Annual Review of Vision Science* **9**, 501–524 (2023).

[12] Poleg-Polsky, A. Dendritic spikes expand the range of well tolerated population noise structures. *Journal of Neuroscience* **39**, 9173–9184 (2019).

[13] Pagkalos, M., Makarov, R. & Poirazi, P. Leveraging dendritic properties to advance machine learning and neuro-inspired computing. *Current Opinion in Neurobiology* **85**, 102853 (2024).

[14] Poirazi, P. & Papoutsi, A. Illuminating dendritic function with computational models. *Nature Reviews Neuroscience* **21**, 303–321 (2020).

[15] Stuart, G. J. & Spruston, N. Dendritic integration: 60 years of progress. *Nature Neuroscience* **18**, 1713–1721 (2015).

[16] Francioni, V. & Harnett, M. T. Rethinking single neuron electrical compartmentalization: dendritic contributions to network computation in vivo. *Neuroscience* **489**, 185–199 (2022).

[17] Stuyt, G., Godenzini, L. & Palmer, L. M. Local and global dynamics of dendritic activity in the pyramidal neuron. *Neuroscience* **489**, 176–184 (2022).

[18] Stingl, M., Draguhn, A. & Both, M. A dendrite is a dendrite is a dendrite? Dendritic signal integration beyond the "antenna" model. *Pflügers Archiv-European Journal of Physiology* **477**, 9–16 (2025).

[19] Zador, A. *et al.* Catalyzing next-generation artificial intelligence through NeuroAI. *Nature Communications* **14**, 1597 (2023).

[20] Iyer, A. *et al.* Avoiding catastrophe: Active dendrites enable multi-task learning in dynamic environments. *Frontiers in Neurorobotics* **16**, 846219 (2022).

[21] Larkum, M. E. Are dendrites conceptually useful? *Neuroscience* **489**, 4–14 (2022).

[22] Rentzeperis, I., Prandi, D. & Bertalmío, M. A neural model for V1 that incorporates dendritic nonlinearities and back-propagating action potentials. *Journal of Neuroscience* **45** (2025).

[23] Jakubovitz, D. & Giryes, R. European Computer Vision Association (ECVA) (ed.) *Improving DNN robustness to adversarial attacks using Jacobian regularization.* (ed.European Computer Vision Association (ECVA)) *European Conference on Computer Vision*, 525–541 (2018).

[24] Tyukin, I., Higham, D. J., Bastounis, A., Woldegeorgis, E. & Gorban, A. N. The feasibility and inevitability of stealth attacks. *IMA Journal of Applied Mathematics* **89**, 44–84 (2024).

[25] Mądry, A., Makelov, A., Schmidt, L., Tsipras, D. & Vladu, A. International Conference on Learning Representations (ed.) *Towards deep learning models resistant to adversarial attacks.* (ed.International Conference on Learning Representations) *Proceedings of the International Conference on Learning Representations* (2018).

[26] Pomponi, J., Scardapane, S. & Uncini, A. Pixle: a fast and effective black-box attack based on rearranging pixels. International Joint Conference on Neural Networks (IJCNN 2022).

[27] Hornik, K., Stinchcombe, M., White, H. *et al.* Multilayer feedforward networks are universal approximators. *Neural networks* **2**, 359–366 (1989).

[28] Cybenko, G. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems* **2**, 303–314 (1989).

[29] Feldman, V. & Zhang, C. NIPS Foundation (ed.) *What neural networks memorize and why: Discovering the long tail via influence estimation.* (ed.NIPS Foundation) *Advances in Neural Information Processing Systems*, Vol. 33, 2881–2891 (2020).

[30] Garg, I., Ravikumar, D. & Roy, K. IMLS (ed.) *Memorization through the lens of curvature of loss function around samples.* (ed.IMLS) *International Conference on Machine Learning*, Vol. 235, 15083–15101 (2024).

[31] Bertalmío, M. *et al.* Evidence for the intrinsically nonlinear nature of receptive fields in vision. *Scientific reports* **10**, 16277 (2020).

[32] Paiton, D. M. *et al.* Selectivity and robustness of sparse coding networks. *Journal of Vision* **20**, 1–28 (2020).

[33] Rawat, S., Heeger, D. & Martiniani, S. Unconditional stability of a recurrent neural circuit implementing divisive normalization. *Advances in Neural Information Processing Systems* **37**, 14712–14750 (2024).

[34] Bai, S., Kolter, J. Z. & Koltun, V. NIPS Foundation (ed.) *Deep equilibrium models.* (ed.NIPS Foundation) *Advances in Neural Information Processing Systems* (2019).

[35] Khona, M. & Fiete, I. R. Attractor and integrator networks in the brain. *Nature Reviews Neuroscience* **23**, 744–766 (2022).

[36] Luna, R., Serrano-Pedraza, I. & Bertalmío, M. Overcoming the limitations of motion sensor models by considering dendritic computations. *Scientific reports* **15**, 9213 (2025).

[37] Vaswani, A. *et al.* Attention is all you need. *Advances in neural information processing systems* **30** (2017).

[38] Hopfield, J. J. Neurons with graded response have collective computational properties like those of two-state neurons. *Proceedings of the national academy of sciences* **81**, 3088–3092 (1984).

[39] Schölkopf, B. *Causality for machine learning*, 765–804 (Association for Computing Machinery, New York, NY, USA, 2022).

[40] Pearl, J. & Mackenzie, D. *The book of why: the new science of cause and effect* (Basic books, 2018).

[41] Bagul, Y. J. On exponential bounds of hyperbolic cosine. *Bulletin of the International Mathematical Virtual Institute* **8**, 365–367 (2018).

[42] Pinkus, A. Approximation theory of the mlp model in neural networks. *Acta Numerica* **8**, 143—195 (1999).

[43] Paszke, A. *et al.* Pytorch: An imperative style, high-performance deep learning library. Preprint at https://arxiv.org/abs/1912.01703 (2019).

[44] Geng, Z. & Kolter, J. Z. TorchDEQ: A library for deep equilibrium models (2023). URL https://github.com/locuslab/torchdeq.

[45] Xiao, H., Rasul, K. & Vollgraf, R. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. Preprint at https://arxiv.org/abs/1708.07747 (2017).

[46] Netzer, Y. *et al.* NIPS Foundation (ed.) *Reading digits in natural images with unsupervised feature learning.* (ed.NIPS Foundation) *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, Vol. 2011, 7 (2011).

[47] Krizhevsky, A. & Hinton, G. Learning multiple layers of features from tiny images. Tech. Rep., University of Toronto (2009). URL https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

[48] TorchVision: PyTorch's computer vision library. https://github.com/pytorch/vision (2016).

[49] Kim, H. Torchattacks: A PyTorch repository for adversarial attacks. Preprint at https://arxiv.org/abs/2010.01950 (2021).

[50] Biggio, B. & Roli, F. Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition* **84**, 317–331 (2018).

[51] Papernot, N. *et al.* ACM Special Interest Group on Security, Audit and Control (ed.) *Practical black-box attacks against machine learning.* (ed.ACM Special Interest Group on Security, Audit and Control) *Proceedings of the ACM on Asia Conference on Computer and Communications Security*, 506–519 (Association for Computing Machinery, New York, NY, USA, 2017).

Supplementary Information

Creating an IBNN by transferring the weights from an existing SM network already trained for the same task

This section demonstrates that when creating an IBNN to solve a given task, one could avoid the training stage by simply transferring the weights from an existing SM network of analogous architecture that has already been trained for the same task. The proof is formulated for the case of a binary classification task and one-hidden-layer networks, but this should not limit its generality. In addition, we show that the magnitude of the parameter λ used in IBNN has an impact on the accuracy achieved by the network.

Formally, let Ψ be a SM-based binary classifier $\Psi(\mathbf{x}) = H(s(\mathbf{x}))$, which performs a binary decision, embodied by the Heaviside function H, based on the sign of a score $s : \mathbb{R}^D \to \mathbb{R}$ assigned to each input sample $\mathbf{x} \in \mathbb{R}^N$ through a single hidden layer of D SM neurons and a FC-based head according to

$$s(\mathbf{x}) = \mathbf{c}^T \ \mathrm{SM}(\mathbf{x}; \mathrm{M}, \mathbf{b}) - c_0 = \sum_{i=1}^{D} c_i \, \phi(\, y_i(\mathbf{x})\,) - c_0 \,, \tag{S1}$$

with $\mathbf{c} = (c_i)_{i=1}^D \in \mathbb{R}^D$ and $c_0 \in \mathbb{R}$ are the weights of the output layer, and where

$$y_i(\mathbf{x}) = \mathbf{m}_i^T \mathbf{x} - b_i \,, \tag{S2}$$

where $\mathbf{m}_i \in \mathbb{R}^N$ and $b_i \in \mathbb{R}$ are, respectively, the linear filter and the bias for the i-th unit of the hidden layer (see Eq. (1) and Eq. (20)). And let us assume, without any loss of generality, that $\{\mathbf{c}, c_0\}$ has been normalized to make $\mathbf{c}$ a unit vector, that is, $\|\mathbf{c}\|_2 = 1$.

We assume that this network has been trained on the set $\mathcal{T}$ composed of two classes $\mathcal{T}_0$ and $\mathcal{T}_1$ achieving perfect accuracy on it, that is,

$$\Psi(\mathbf{x}^{(t)}) = \begin{cases} 1 \,, \ \forall \mathbf{x}^{(t)} \in \mathcal{T}_1 \\ 0 \,, \ \forall \mathbf{x}^{(t)} \in \mathcal{T}_0 \end{cases} :$$

in other words, $\mathrm{SM}(\cdot; \mathrm{M}, \mathbf{b})$ makes the sets $\mathrm{SM}(\mathcal{T}_1; \mathrm{M}, \mathbf{b})$ and $\mathrm{SM}(\mathcal{T}_0; \mathrm{M}, \mathbf{b})$ linearly separable, and $\{\mathbf{c}, c_0\}$ separates them. Furthermore, we assume that such perfect classifications is achieved through the classifier s of Eq. (S1) by a margin M, that is, that

$$\min_{\mathbf{x}^{(1)} \in \mathcal{T}_1} \left\{ s(\mathbf{x}^{(1)}) \right\} \ - \ \max_{\mathbf{x}^{(0)} \in \mathcal{T}_0} \left\{ s(\mathbf{x}^{(0)}) \right\} \ \geq \ M \,. \tag{S3}$$

We define another network $\hat{\Psi}(\mathbf{x}) = H(\hat{s}(\mathbf{x}))$, with analogous architecture to Ψ but with IBNN instead of SM neurons in its hidden layer, and with the peculiarity that the IBNN neurons use the same exact $\{\mathrm{M}, \mathbf{b}\}$ of the respective SM neurons in Eqs. (S1)-(S2) but whose scoring hyperplane $\{\hat{\mathbf{c}}, \hat{c}_0\}$ is allowed to differ, and thus

$$\hat{s}(\mathbf{x}) = \hat{\mathbf{c}}^T \, \mathrm{IBNN}(\mathbf{x}; \mathrm{M}, \mathbf{b}, \lambda) - \hat{c}_0 = \sum_{i=1}^{D} \hat{c}_i \, \phi(\, z_i(\mathbf{x})\,) - \hat{c}_0 \,, \tag{S4}$$

where

$$z_i(\mathbf{x}) = \mathbf{m}_i^T \mathbf{x} - b_i \ - \ \lambda \sum_{k=1}^{D} w_{ik} \, \sigma(z_k(\mathbf{x}) - z_i(\mathbf{x})) \,. \tag{S5}$$

We will show that the training error achievable by the IBNN-based classifier $\hat{\Psi}$ is below a bound that depends linearly on the magnitude $|\lambda|$ and which could be made, consequently, arbitrarily small. We will do so by studying the case of $\hat{\mathbf{c}} = \mathbf{c}$ and $\hat{c}_0 = c_0$, that is, a scoring hyperplane for $\hat{\Psi}$ identical to that of Ψ, as a suboptimal instance of $\{\hat{\mathbf{c}}, \hat{c}_0\}$. In the remaining of the section we will use, for convenience, the shorter notation $\mathrm{IBNN}(\mathbf{x}) = \mathrm{IBNN}(\mathbf{x}; \mathrm{M}, \mathbf{b}, \lambda)$ and $\mathrm{SM}(\mathbf{x}) = \mathrm{SM}(\mathbf{x}; \mathrm{M}, \mathbf{b})$ where the parameters of the layer are left implied.

First, a bound on the maximum distance between the images of the maps $\mathrm{SM}(\cdot)$ and $\mathrm{IBNN}(\cdot)$ for the same data point $\mathbf{x}$ can be established. The difference can be written as

$$\mathrm{IBNN}(\mathbf{x}) - \mathrm{SM}(\mathbf{x}) = (\Phi \circ \mathrm{F} \circ \mathrm{A})(\mathbf{x}) - (\Phi \circ \mathrm{A})(\mathbf{x}) = (\Phi \circ \mathrm{F})(\mathrm{A}(\mathbf{x})) - \Phi(\mathrm{A}(\mathbf{x}))\,, \tag{S6}$$

where the dependence $\mathrm{F} = \mathrm{F}(\cdot; \lambda)$ has also left implied. From that form, for any $\Phi = (\phi)_{i=1}^D$ contractive like in the case of $\phi = \mathrm{ReLU}$, we have that, for any $\mathbf{y} = \mathrm{A}(\mathbf{x})$,

$$\begin{aligned}\|(\Phi\circ \mathrm{F})(\mathbf{y})-\Phi(\mathbf{y})\|_2^2 &\leq \|\mathrm{F}(\mathbf{y})-\mathbf{y}\|_2^2\\ &= \|-\lambda\,\mathrm{B}(\mathrm{F}(\mathbf{y}))\|_2^2 \;\leq\; |\lambda|^2\,\|\,\mathrm{B}(\mathrm{F}(\mathbf{y}))\,\|_2^2\\ &\leq |\lambda|^2\,D\,,\end{aligned} \tag{S7}$$

wherein the second line is derived from the very definition of the fixed-point function F (see Eq. (17)) when a unique solution exists, since $\mathrm{F}(\mathbf{y})=\mathbf{y}-\lambda\mathrm{B}(\mathrm{F}(\mathbf{y}))$, and wherein the last inequality derives straightforwardly from the definition of $\mathrm{B}=(\mathrm{B}_i)_{i=1}^D$ (see Eq. (16)) for a sigmoidal $|\sigma|\leq 1$ and positive and normalized w_{ik} as

$$|\mathrm{B}_i(\mathbf{z})| \;=\; \left|\sum_{k=1}^{D} w_{ik}\sigma(z_k-z_i)\right| \;\leq\; \sum_{k=1}^{D}|w_{ik}|\,|\sigma(z_k-z_i)| \;\leq\; \sum_{k=1}^{D}|w_{ik}|\leq 1\,. \tag{S8}$$

Therefore, since by virtue of Eqs. (S6)-(S7)

$$\|\mathrm{IBNN}(\mathbf{x})-\mathrm{SM}(\mathbf{x})\|_2 \;\leq\; |\lambda|\sqrt{D}\,, \tag{S9}$$

we can write, for any $\mathbf{x}\in\mathbb{R}^N$,

$$\mathrm{IBNN}(\mathbf{x})=\mathrm{SM}(\mathbf{x})+\alpha(\mathbf{x})\,\mathbf{q}(\mathbf{x}) \tag{S10}$$

for certain $\alpha(\mathbf{x})\in\mathbb{R}$ and $\mathbf{q}(\mathbf{x})\in\mathbb{R}^D$ such that $0\leq\alpha(\mathbf{x})\leq|\lambda|\sqrt{D}$ and $\|\mathbf{q}(\mathbf{x})\|_2=1$. Using this fact, and for an identical scoring hyperplane to that of Ψ, that is, $\hat{\mathbf{c}}=\mathbf{c}$ and $\hat{c}_0=c_0$), we have, $\forall\mathbf{x}^{(1)}\in\mathcal{T}_1$ and $\forall\mathbf{x}^{(0)}\in\mathcal{T}_0$,

$$\begin{aligned}&\hat{s}(\mathbf{x}^{(1)})-\hat{s}(\mathbf{x}^{(0)})\\ &= \Big(\mathbf{c}^T\,\mathrm{IBNN}(\mathbf{x}^{(1)})-c_0\Big)\;-\;\Big(\mathbf{c}^T\,\mathrm{IBNN}(\mathbf{x}^{(0)})-c_0\Big)\\ &= \Big(\mathbf{c}^T\,\Big(\mathrm{SM}(\mathbf{x}^{(1)})+\alpha(\mathbf{x}^{(1)})\,\mathbf{q}(\mathbf{x}^{(1)})\Big)-c_0\Big)-\Big(\mathbf{c}^T\,\Big(\mathrm{SM}(\mathbf{x}^{(0)})+\alpha(\mathbf{x}^{(0)})\,\mathbf{q}(\mathbf{x}^{(0)})\Big)-c_0\Big)\\ &= \Big(\Big(\mathbf{c}^T\,\mathrm{SM}(\mathbf{x}^{(1)})-c_0\Big)-\Big(\mathbf{c}^T\,\mathrm{SM}(\mathbf{x}^{(0)})-c_0\Big)\Big)+\mathbf{c}^T\,\Big(\alpha(\mathbf{x}^{(1)})\,\mathbf{q}(\mathbf{x}^{(1)})-\alpha(\mathbf{x}^{(0)})\,\mathbf{q}(\mathbf{x}^{(0)})\Big)\\ &\geq\; M-2|\lambda|\sqrt{D}\,,\end{aligned} \tag{S11}$$

where the last inequality results from the assumption of Eq. (S3) for a margin M in the SM-based classifier and from the worst case alignment between the unit vectors $\mathbf{c}$ and $\mathbf{q}(\mathbf{x}^{(i)})$.